

입찰제안서 / RFP 파싱 전환 — 내부망 적용 지시서

소스를 들고 들어갈 수 없는 환경을 전제로, 이 문서만 보고 손으로 고칠 수 있게 파일별 변경 내용을 모두 실었다

작성일	2026-09-23
적용 대상	내부망 uploader — 파싱 전환 이전 상태 (외부망 2ebfcda 시점과 같다)
목표 상태	외부망 GGReportAgent/uploader @ 1515821 와 동일
표기	+ 로 시작하는 초록 줄 = 넣을 줄 · - 로 시작하는 붉은 줄 = 지울 줄 · 나머지는 위치를 잡기 위한 주변 줄

이 문서만으로 끝나지 않는 것이 셋 있다. 손으로 옮겨 적을 수 없는 분량이라 반입이 필요하다.
① 라이브러리 jar 4종(+전이 의존) ② parse 패키지 자바 10개(약 1,400줄) ③ agent-test.js(577줄). 자세한 목록은 2장에 있다.
이 셋을 뺀 나머지 38개 파일은 이 문서의 4장~10장에 전부 실려 있다.

1. 무엇이 바뀌는가

분류 방식이 통째로 바뀐다. 파일명 규칙으로 기관을 알아내 classified/ 로 파일을 옮기던 것을, 문서 내용을 파싱해 산출물을 만드는 방식으로 교체한다. 원본은 옮기지 않고 userdata/ 에 보관한다.

구분	변경 전 (지금 내부망)	변경 후
업로드	pdf:hwp-md, 파일명 년도_기관명_설명	제안서 ppt:pptx / RFP pdf:hwp-hwp, 창구 2개
기관 판별	파일명에서 잘라 TSKGIAF02 조회	문서 본문에서 찾아 매칭, 없으면 미분류
결과물	원본을 classified/ 로 이동	제안서 → {연도}_{기관분류}_{기관명}_{파싱일}_bid_proposal.json RFP → {공고일}_{기관분류}_{기관명}_{파싱일}.md
상태	분류상태 1개 축	분류상태 + 파싱상태 2개 축
재처리	5분 주기 배치	배치 폐지 → 화면의 수동 재파싱
연동	/api/files/*	위 유지 + /api/proposals, /api/rfps, agent 테스트 패널

적용 순서

- DDL 먼저 (3장). 소스를 먼저 올리면 조회에서 ORA-00904 가 난다.
- 반입물 배치 (2장) — jar, parse 패키지, agent-test.js
- 설정 → 도메인·코드값 → 매퍼 → 서비스 → 컨트롤러 → 화면 (4~9장)
- 폐기 파일 삭제 (10장) · 테스트 정리 (9장)
- 빌드 → 기동 → 검증 (11장)

내부망 고유 자산은 그대로 둔다. 감사 2열(시스템사용일시 · 시스템사용자번호), SystemUser ThreadLocal, databaseId 오라클/MySQL 분기, 화면번호 규약 (KGI#####이름), 스케줄러 추상화, 삭제·반려 화면, 검색 API 는 손대지 않는다. 아래 diff 에도 그 부분은 나오지 않는다.

2. 반입이 필요한 것

2-1. 라이브러리 jar

groupId:artifactId	버전	쓰는 곳
org.apache.poi:poi-scratchpad	5.2.3	.ppt(97-2003) 슬라이드 추출
kr.dogfoot:hwplib	1.1.11	HWP 본문·표
kr.dogfoot:hwplib	1.0.9	HWPX 본문·표
org.apache.lucene:lucene-analyzers-nori	8.11.4	한국어 형태소 분석(요약)

⚠ 전이 의존 `org.apache.lucene:lucene-core:8.11.4` 를 빠뜨리기 쉽다. 외부망에서 `mvn dependency:copy-dependencies` 로 모아 확인할 것. pdfbox 2.0.29 · poi-ooxml 5.2.3 은 이미 있으므로 반입 대상이 아니다.

2-2. 자바 소스 — `com.kb.uploader.parse` 패키지 10개

문서를 읽고 요약하는 계층이다. DB·스프링 설정에 의존하지 않아(애노테이션은 `@Component` 뿐) 내부망에서 고칠 것이 없다. 파일을 그대로 넣으면 된다.

파일	줄수	역할
<code>PptExtractor.java</code>	87	ppt-pptx → 슬라이드별 제목·본문·이미지 유무
<code>RfpExtractor.java</code>	178	pdf-hwp-hwp → 문단/표 블록
<code>ExtractedDocument.java</code>	65	추출 문서 모델
<code>DocBlock.java</code>	39	문단/표 블록
<code>SlideContent.java</code>	31	슬라이드 1장
<code>ProposalDocument.java</code>	42	제안서 JSON 스키마
<code>InstitutionMatcher.java</code>	141	본문 ↔ 기관 목록 매칭
<code>DateDetector.java</code>	117	제안서 연도 · RFP 공고일 추정
<code>TextRankSummarizer.java</code>	118	Nori + PageRank 추출 요약
<code>RfpSummaryWriter.java</code>	606	요약 Markdown 생성

2-3. 정적 자원 — `static/js/agent-test.js` (577줄)

agent 테스트 패널. 외부 라이브러리·CDN 을 쓰지 않는 순수 JS 라 그대로 넣으면 된다. 이 파일이 없어도 나머지 기능은 정상이므로, 반입이 어려우면 패널만 빼고 진행해도 된다 (그 경우 `layout.html` 의 agent 관련 블록도 함께 빼야 한다).

3. DB 작업 — 가장 먼저

마이그레이션 DDL 두 벌이 저장소에 있다. 내부망 Oracle 은 `-oracle.sql`, 외부망 MySQL 은 `-mysql.sql`.

```
uploader/src/main/resources/migration/2026-09-23_TSKGIAF01_파싱컬럼추가-oracle.sql
uploader/src/main/resources/migration/2026-09-23_TSKGIAF01_파싱컬럼추가-mysql.sql
```

추가 컬럼	타입	값
문서종류구분	VARCHAR2(2 CHAR)	01 입찰제안서 / 02 RFP
파싱상태구분	VARCHAR2(2 CHAR)	01 PENDING / 02 SUCCESS / 03 FAILED
파싱메시지내용	VARCHAR2(1000 CHAR)	실패 사유
파싱산출물경로내용	VARCHAR2(600 CHAR)	JSON·MD 경로
파싱일시	VARCHAR2(14 CHAR)	YYYYMMDDHH24MISS
문서페이지수	DECIMAL(5)	슬라이드/페이지 수
문서일자	VARCHAR2(8 CHAR)	연도 4자리 또는 공고일 8자리

한글 컬럼명 인코딩 사고가 실제로 있었다(2026-09-22). 클라이언트 charset 이 어긋난 채 실행해 컬럼명이 모지 바케로 만들어졌고 앱이 `Unknown column '분류상태구분'` 로 죽었다. 복구 불가능한 바이트가 섞여 이름만 고칠 수도 없어 테이블을 다시 만들어야 했다. **SQL Developer·Workbench** 처럼 **UTF-8 이 기본인 도구로 실행하고, DDL 안의 검증 쿼리를 반드시 돌릴 것.**

컬럼 수를 줄이는 절충안이 검토 대기 중이다. 7개 중 5개(문서종류구분·파싱산출물경로내용·문서일자· 파싱일시· 문서페이지수)는 기존 컬럼과 산출물 파일명으로 대체할 수 있다. DBA 제출 전에 `TODO_modify.md` 의 "내부망 스키마 변경 폭 줄이기" 항목을 먼저 읽고 최종안을 정할 것. (권장안: **파싱상태구분 + 파싱메시지내용 2개만 추가**)

4. 설정

uploader/pom.xml

의존성 4개 추가 반입한 jar 를 Maven 이 찾게 한다. 전이 의존(lucene-core 8.11.4)도 함께 반입해야 한다.

@@ -66,8 +66,34 @@

```
        <artifactId>pdfbox</artifactId>
        <version>2.0.29</version>
    </dependency>
    <!-- Apache POI (XLSX) -->
+   <!-- — 2026-09-23 파싱 전환으로 추가 (폐쇄망 반입 시 전이 의존까지 함께 반입할 것) — -->
+
+   <!-- .ppt(97-2003) 슬라이드 추출 -->
+
+   <dependency>
+
+       <groupId>org.apache.poi</groupId>
+
+       <artifactId>poi-scratchpad</artifactId>
+
+       <version>5.2.3</version>
+
+   </dependency>
+
+   <!-- HWP 본문·표 -->
+
+   <dependency>
+
+       <groupId>kr.dogfoot</groupId>
+
+       <artifactId>hwplib</artifactId>
+
+       <version>1.1.11</version>
+
+   </dependency>
+
+   <!-- HWPX 본문·표 -->
+
+   <dependency>
+
+       <groupId>kr.dogfoot</groupId>
+
+       <artifactId>hwpplib</artifactId>
+
+       <version>1.0.9</version>
+
+   </dependency>
+
+   <!-- 한국어 형태소 분석 (TextRank 요약). lucene-core 8.11.4 가 전이 의존으로 따라온다 -->
+
+   <dependency>
+
+       <groupId>org.apache.lucene</groupId>
+
+       <artifactId>lucene-analyzers-nori</artifactId>
+
+       <version>8.11.4</version>
+
+   </dependency>
+
+
+   <dependency>
```

```
<groupId>org.apache.poi</groupId>
<artifactId>poi-ooxml</artifactId>
<version>5.2.3</version>
```

uploader/config-envs/dev/application.properties

환경 설정 (dev) 5개 환경 파일에 같은 블록을 넣는다. 한 곳이라도 빠지면 그 환경에서 FileStorageService 가 기동하다 죽는다.

```
@@ -12,7 +12,21 @@ upload.base-dir=/app/uploader

mybatis.mapper-locations=classpath:mapper/*.xml
mybatis.type-aliases-package=com.kb.uploader.domain
mybatis.configuration.map-underscore-to-camel-case=true

-reclassification.cron=0 */5 * * * *

+# (2026-09-23 배치 폐지) reclassification.cron=0 */5 * * * *

logging.level.org.springframework.web.filter.CommonsRequestLoggingFilter=DEBUG
+

+# ---- 2026-09-23 파싱 전환: 원본 보관 / 산출물 경로 ----

+# .properties 는 ISO-8859-1 로 읽히므로 한글 경로는 \uXXXX 로 적는다.

+# \uc6d0\ubcf8/\... = 원본/전처리/입찰제안서_PPT

+# \uc804\ucc98\uub9ac/RFP/\... = 전처리/RFP/파싱후

+upload.userdata-dir=${upload.base-dir}/userdata

+upload.proposal-json-dir=${upload.base-dir}/\uc6d0\ubcf8/\uc804\ucc98\uub9ac/\uc785\ucc30\uc81c\uc548\uc11c_PPT

+upload.rfp-md-dir=${upload.base-dir}/\uc804\ucc98\uub9ac/RFP/\ud30c\uc2f1\u6c4

+

+# 한글 품값·파일명 인코딩 고정

+server.servlet.encoding.charset=UTF-8

+server.servlet.encoding.enabled=true

+server.servlet.encoding.force=true

+server.tomcat.uri-encoding=UTF-8
```

uploader/config-envs/local/application.properties

```
@@ -12,7 +12,21 @@ upload.base-dir=C:/uploader-local

mybatis.mapper-locations=classpath:mapper/*.xml
mybatis.type-aliases-package=com.kb.uploader.domain
mybatis.configuration.map-underscore-to-camel-case=true

-reclassification.cron=0 */5 * * * *

+# (2026-09-23 배치 폐지) reclassification.cron=0 */5 * * * *

logging.level.org.springframework.web.filter.CommonsRequestLoggingFilter=DEBUG
+

+# ---- 2026-09-23 파싱 전환: 원본 보관 / 산출물 경로 ----
```

```
## .properties 는 ISO-8859-1 로 읽히므로 한글 경로는 \uXXXX 로 적는다.
```

```
## \uc6d0\ubcf8/\... = 원본/전처리/입찰제안서_PPT
```

```
## \uc804\ucc98\uub9ac/RFP/\... = 전처리/RFP/파싱후
```

```
+upload.userdata-dir=${upload.base-dir}/userdata
```

```
+upload.proposal-json-dir=${upload.base-dir}/\uc6d0\ubcf8/\uc804\ucc98\uub9ac/\uc785\ucc30\ucc81c\ucc548\ucc11c_PPT
```

```
+upload.rfp-md-dir=${upload.base-dir}/\uc804\ucc98\uub9ac/RFP/\ud30c\uc2f1\udd6c4
```

```
+
```

```
## 한글 품값·파일명 인코딩 고정
```

```
+server.servlet.encoding.charset=UTF-8
```

```
+server.servlet.encoding.enabled=true
```

```
+server.servlet.encoding.force=true
```

```
+server.tomcat.uri-encoding=UTF-8
```

uploader/config-envs/out-local/application.properties

```
@@ -14,7 +14,21 @@ upload.base-dir=C:/uploader-out-local
```

```
mybatis.mapper-locations=classpath:mapper/*.xml  
mybatis.type-aliases-package=com.kb.uploader.domain  
mybatis.configuration.map-underscore-to-camel-case=true
```

```
-reclassification.cron=0 */5 * * * *
```

```
## (2026-09-23 배치 폐지) reclassification.cron=0 */5 * * * *
```

```
logging.level.org.springframework.web.filter.CommonsRequestLoggingFilter=DEBUG
```

```
+
```

```
## ---- 2026-09-23 파싱 전환: 원본 보관 / 산출물 경로 ----
```

```
## .properties 는 ISO-8859-1 로 읽히므로 한글 경로는 \uXXXX 로 적는다.
```

```
## \uc6d0\ubcf8/\... = 원본/전처리/입찰제안서_PPT
```

```
## \uc804\ucc98\uub9ac/RFP/\... = 전처리/RFP/파싱후
```

```
+upload.userdata-dir=${upload.base-dir}/userdata
```

```
+upload.proposal-json-dir=${upload.base-dir}/\uc6d0\ubcf8/\uc804\ucc98\uub9ac/\uc785\ucc30\ucc81c\ucc548\ucc11c_PPT
```

```
+upload.rfp-md-dir=${upload.base-dir}/\uc804\ucc98\uub9ac/RFP/\ud30c\uc2f1\udd6c4
```

```
+
```

```
## 한글 품값·파일명 인코딩 고정
```

```
+server.servlet.encoding.charset=UTF-8
```

```
+server.servlet.encoding.enabled=true
```

```
+server.servlet.encoding.force=true
```

```
+server.tomcat.uri-encoding=UTF-8
```

uploader/config-envs/prod/application.properties

```
@@ -5,5 +5,19 @@ upload.base-dir=/app/uploader

mybatis.mapper-locations=classpath:mapper/*.xml
mybatis.type-aliases-package=com.kb.uploader.domain
mybatis.configuration.map-underscore-to-camel-case=true

-reclassification.cron=0 */5 * * * *

+# (2026-09-23 배치 폐지) reclassification.cron=0 */5 * * * *
+
+# ---- 2026-09-23 파싱 전환: 원본 보관 / 산출물 경로 ----
+
+# .properties 는 ISO-8859-1 로 읽히므로 한글 경로는 \uXXXX 로 적는다.
+# \uc6d0\ubcf8/\... = 원본/전처리/입찰제안서_PPT
+# \uc804\ucc98\uub9ac/RFP/\... = 전처리/RFP/파싱후
+upload.userdata-dir=${upload.base-dir}/userdata
+upload.proposal-json-dir=${upload.base-dir}/\uc6d0\ubcf8/\uc804\ucc98\uub9ac/\uc785\ucc30\uc81c\uc548\uc11c_PPT
+upload.rfp-md-dir=${upload.base-dir}/\uc804\ucc98\uub9ac/RFP/\ud30c\uc2f1\uud6c4
+
+# 한글 품값·파일명 인코딩 고정
+server.servlet.encoding.charset=UTF-8
+server.servlet.encoding.enabled=true
+server.servlet.encoding.force=true
+server.tomcat.uri-encoding=UTF-8
```

uploader/config-envs/stg/application.properties

```
@@ -12,5 +12,19 @@ upload.base-dir=/app/uploader

mybatis.mapper-locations=classpath:mapper/*.xml
mybatis.type-aliases-package=com.kb.uploader.domain
mybatis.configuration.map-underscore-to-camel-case=true

-reclassification.cron=0 */5 * * * *

+# (2026-09-23 배치 폐지) reclassification.cron=0 */5 * * * *
+
+# ---- 2026-09-23 파싱 전환: 원본 보관 / 산출물 경로 ----
+
+# .properties 는 ISO-8859-1 로 읽히므로 한글 경로는 \uXXXX 로 적는다.
+# \uc6d0\ubcf8/\... = 원본/전처리/입찰제안서_PPT
+# \uc804\ucc98\uub9ac/RFP/\... = 전처리/RFP/파싱후
+upload.userdata-dir=${upload.base-dir}/userdata
+upload.proposal-json-dir=${upload.base-dir}/\uc6d0\ubcf8/\uc804\ucc98\uub9ac/\uc785\ucc30\uc81c\uc548\uc11c_PPT
+upload.rfp-md-dir=${upload.base-dir}/\uc804\ucc98\uub9ac/RFP/\ud30c\uc2f1\uud6c4
```

```
+  
## 한글 품값·파일명 인코딩 고정  
+server.servlet.encoding.charset=UTF-8  
+server.servlet.encoding.enabled=true  
+server.servlet.encoding.force=true  
+server.tomcat.uri-encoding=UTF-8
```

uploader/src/test/resources/application.properties

```
@@ -23,5 +23,19 @@ mybatis.mapper-locations=classpath:mapper/*.xml  
  
mybatis.type-aliases-package=com.kb.uploader.domain  
mybatis.configuration.map-underscore-to-camel-case=true  
  
upload.base-dir=/tmp/uploader-test  
-reclassification.cron=0 */5 * * * *  
  
## (2026-09-23 배치 폐지) reclassification.cron=0 */5 * * * *  
  
+  
## ---- 2026-09-23 파싱 전환: 원본 보관 / 산출물 경로 ----  
  
## .properties 는 ISO-8859-1 로 읽히므로 한글 경로는 \uXXXX 로 적는다.  
  
## \uc6d0\ubcf8/\... = 원본/전처리/입찰제안서_PPT  
  
## \uc804\ucc98\uub9ac/RFP/\... = 전처리/RFP/파싱후  
  
+upload.userdata-dir=${upload.base-dir}/userdata  
  
+upload.proposal-json-dir=${upload.base-dir}/\uc6d0\ubcf8/\uc804\ucc98\uub9ac/\uc785\ucc30\ucc81c\ucc548\ucc11c_PPT  
  
+upload.rfp-md-dir=${upload.base-dir}/\uc804\ucc98\uub9ac/RFP/\ud30c\uc2f1\udd6c4  
  
+  
## 한글 품값·파일명 인코딩 고정  
+server.servlet.encoding.charset=UTF-8  
+server.servlet.encoding.enabled=true  
+server.servlet.encoding.force=true  
+server.tomcat.uri-encoding=UTF-8
```

5. 도메인 · 코드값 · TypeHandler

domain/UploadedFile.java

도메인 필드 7개 DB 컬럼 7개에 대응한다. 기존 year(문서년)는 옛 행이 쓰므로 남겨 두고 새 값은 docDate 에 넣는다.

```
@@ -26,10 +26,100 @@ public class UploadedFile {

    private String status;
    private LocalDateTime uploadedAt;
    private LocalDateTime classifiedAt;

+   // — 2026-09-23 파싱 전환으로 추가한 7개 —————
+   /** 문서종류구분. code/DocumentType 의 이름값(BID_PROPOSAL·RFP). DB 에는 코드(01·02)로 들어간다. */
+   private String docType;
+   /** 파싱상태구분. code/ParseStatus 의 이름값(PENDING·SUCCESS·FAILED). DB 에는 코드(01·02·03). */
+   private String parseStatus;
+   /** 파싱 실패 사유. 화면의 실패 배지 옆에 그대로 보여 준다. */
+   private String parseMessage;
+   /** 산출물(제안서 JSON·RFP 요약 MD) 경로. ⚠ 원본 경로인 storedPath 와 다르다. */
+   private String outputPath;
+   /** 제안서 슬라이드 수 / RFP 페이지 수. */
+   private Integer pageCount;
+   private LocalDateTime parsedAt;
+   /**
+   * 제안서는 연도(yyyy), RFP 는 공고일(yyyyMMdd).
+   * ⚠ 기존 {@code year}(문서년 VARCHAR2(4))는 8자리를 담지 못해 새 컬럼을 쓴다.
+   * 옛 행이 문서년을 쓰고 있으므로 그 필드는 남겨 두고 <b>새 코드는 이 값만</b> 쓴다.
+   */
+   private String docDate;
+
+   /** 기관 테이블에 없을 때의 기관분류 값. ⚠ InstitutionCategory 코드에는 없는 값이다. */
+   public static final String UNKNOWN_CATEGORY = "미분류";
+
    public UploadedFile() {}

+   /** 파싱 전환 후의 업로드. 기관·연도는 파싱해서 채우므로 이 시점에는 모른다. */
+   public UploadedFile(String docType, String originalName, String storedPath) {
+       this.docType = docType;
+       this.originalName = originalName;
```

```

+     this.storedPath = storedPath;
+
+     this.status = "UNCLASSIFIED";
+
+     this.parseStatus = "PENDING";
+
+     this.uploadedAt = LocalDateTime.now();
+ }
+
+ /**
+  * 파싱 성공 기록.
+  *
+  * <p>⚠ 파싱상태와 분류상태는 <b>다른 축</b>이다. 파싱은 됐는데 기관 테이블에 없어
+  * 분류가 안 되는 건이 정상적으로 존재한다 - 그 경우 분류상태는 UNCLASSIFIED 로 남는다.
+  */
+
+ public void markParsed(String institutionName, String category, String docDate,
+
+     String outputPath, Integer pageCount, String message) {
+
+     this.institutionName = institutionName;
+
+     this.category = category;
+
+     this.docDate = docDate;
+
+     this.outputPath = outputPath;
+
+     this.pageCount = pageCount;
+
+     this.parseMessage = truncate(message);
+
+     this.parseStatus = "SUCCESS";
+
+     this.parsedAt = LocalDateTime.now();
+
+     if (category != null && !UNKNOWN_CATEGORY.equals(category)) {
+
+         this.status = "CLASSIFIED";
+
+         this.classifiedAt = this.parsedAt;
+
+     } else {
+
+         this.status = "UNCLASSIFIED";
+
+         this.classifiedAt = null;
+
+     }
+ }
+
+
+
+ public void markParseFailed(String message) {
+
+     this.parseStatus = "FAILED";
+
+     this.parseMessage = truncate(message);
+
+     this.parsedAt = LocalDateTime.now();

```

```

+         this.status = "UNCLASSIFIED";
+     }
+
+
+     /** 파싱메시지내용 은 1000자다. 넘치면 잘라 넣는다(길이 초과로 INSERT 가 죽는 쪽이 더 나쁘다). */
+     private static String truncate(String message) {
+         if (message == null) return null;
+
+         return message.length() <= 1000 ? message : message.substring(0, 997) + "...";
+     }
+
+
+     /**
+      * 화면에 보여 줄 기관분류. DB 에는 "미분류" 코드가 없어 NULL 로 저장하므로
+      * 읽을 때 되돌린다(Mapper XML 의 {@code 기관분류값} 조각 참조).
+      */
+
+     public String getCategoryLabel() {
+         return category == null ? UNKNOWN_CATEGORY : category;
+     }
+
+
+     /** 문서종류 표시명. 옛 행은 null 이라 "이전 방식" 으로 보여 준다. */
+
+     public String getDocTypeLabel() {
+         return docType == null ? "이전 방식" : com.kb.uploader.code.DocumentType.label(docType);
+     }
+
+
+     /** 산출물 파일명만. 화면 표시용. */
+
+     public String getOutputFileName() {
+         if (outputPath == null) return null;
+
+         int slash = Math.max(outputPath.lastIndexOf('/'), outputPath.lastIndexOf('\\'));
+
+         return slash < 0 ? outputPath : outputPath.substring(slash + 1);
+     }
+
+
+
+
+     public UploadedFile(String originalName, String storedPath,
+                         String year, String institutionName) {
+         this.originalName = originalName;
+         this.storedPath = storedPath;
+
+@@ -72,5 +162,20 @@ public class UploadedFile {
+
+     public String getSystemUserNo() { return systemUserNo; }
+     public void setSystemUserNo(String systemUserNo) { this.systemUserNo = systemUserNo; }
+     public LocalDateTime getSystemUsedAt() { return systemUsedAt; }
+     public void setSystemUsedAt(LocalDateTime systemUsedAt) { this.systemUsedAt = systemUsedAt; }

```

```

+
+ public String getDocType() { return docType; }
+ public void setDocType(String docType) { this.docType = docType; }
+ public String getParseStatus() { return parseStatus; }
+ public void setParseStatus(String parseStatus) { this.parseStatus = parseStatus; }
+ public String getParseMessage() { return parseMessage; }
+ public void setParseMessage(String parseMessage) { this.parseMessage = parseMessage; }
+ public String getOutputPath() { return outputPath; }
+ public void setOutputPath(String outputPath) { this.outputPath = outputPath; }
+ public Integer getPageCount() { return pageCount; }
+ public void setPageCount(Integer pageCount) { this.pageCount = pageCount; }
+ public LocalDateTime getParsedAt() { return parsedAt; }
+ public void setParsedAt(LocalDateTime parsedAt) { this.parsedAt = parsedAt; }
+ public String getDocDate() { return docDate; }
+ public void setDocDate(String docDate) { this.docDate = docDate; }
+
+ }

```

code/DocumentType.java

문서종류 코드값 (신규) ClassificationStatus 와 같은 모양. 확장자 판정과 화면 표시명도 여기 둔다.

```

new file mode 100644
@@ -0,0 +1,76 @@

+package com.kb.uploader.code;
+
+import java.util.Arrays;
+import java.util.Collections;
+import java.util.HashSet;
+import java.util.LinkedHashMap;
+import java.util.Map;
+import java.util.Set;
+
+/**
+ * 문서종류 - 사내 인포타임 {@code 구분코드2} 로 저장하는 코드값 (2026-09-23 신설).
+ *
+ * <p>자바가 쓰는 값은 {@code "BID_PROPOSAL"}·{@code "RFP"} 이고, DB 에는
+ * {@code '01'}·{@code '02'} 로 들어간다. 변환은 {@code DocumentTypeHandler} 한 곳뿐이다
+ * - {@link ClassificationStatus} 와 같은 방식이다.

```

+ *

+ * <p>⚠ Mapper XML 의 SQL 리터럴은 이 변환을 타지 않는다.</p>

+ * {@code WHERE 문서종류구분 = '01'} 처럼 코드로 적어야 한다.

+ * 그 자리에 {@code 'BID_PROPOSAL'} 을 적으면 오류 없이 0건이 나온다.

+ *

+ * <p>확장자 판정과 화면 표시명도 여기 둔다. 문서 종류를 늘릴 때 고칠 곳이 한 군데가 되게 하려는 것이다.

+ */

+public final class DocumentType {

+

+ public static final String BID_PROPOSAL = "BID_PROPOSAL";

+ public static final String RFP = "RFP";

+

+ private static final Map<String, String> NAME_TO_CODE = new LinkedHashMap<String, String>();

+ private static final Map<String, String> CODE_TO_NAME = new LinkedHashMap<String, String>();

+ private static final Map<String, String> LABEL = new LinkedHashMap<String, String>();

+ private static final Map<String, Set<String>> EXTENSIONS = new LinkedHashMap<String, Set<String>>();

+

+ static {

+ put("01", BID_PROPOSAL, "입찰제안서", "ppt", "pptx");

+ put("02", RFP, "입찰공고문(RFP)", "pdf", "hwp", "hwp");

+ }

+

+ private static void put(String code, String name, String label, String... extensions) {

+ NAME_TO_CODE.put(name, code);

+ CODE_TO_NAME.put(code, name);

+ LABEL.put(name, label);

+ EXTENSIONS.put(name, Collections.unmodifiableSet(new HashSet<String>(Arrays.asList(extensions))));

+ }

+

+ public static Map<String, String> nameToCode() {

+ return Collections.unmodifiableMap(NAME_TO_CODE);

+ }

+

+ public static Map<String, String> codeToName() {

+ return Collections.unmodifiableMap(CODE_TO_NAME);

```

+   }
+
+   /** 화면 표시명. 모르는 값이면 그대로 돌려준다(옛 행은 문서종류가 null 이다). */
+   public static String label(String name) {
+       String label = LABEL.get(name);
+       return label != null ? label : name;
+   }
+
+
+   public static Set<String> extensions(String name) {
+       Set<String> exts = EXTENSIONS.get(name);
+       return exts != null ? exts : Collections.<String>emptySet();
+   }
+
+
+   /** 이 종류가 받아들이는 확장자인가. */
+   public static boolean accepts(String name, String filename) {
+       return extensions(name).contains(extensionOf(filename));
+   }
+
+
+   public static String extensionOf(String filename) {
+       if (filename == null) return "";
+       int dot = filename.lastIndexOf('.');
+       return dot < 0 ? "" : filename.substring(dot + 1).toLowerCase();
+   }
+
+
+   private DocumentType() {}
+
+}

```

code/ParseStatus.java

파싱상태 코드값 (신규) 분류상태와 다른 축이다. 01 PENDING / 02 SUCCESS / 03 FAILED.

```

new file mode 100644
@@ -0,0 +1,45 @@
+package com.kb.uploader.code;
+
+import java.util.Collections;
+import java.util.LinkedHashMap;

```

```
+import java.util.Map;
+
+/**
+ * 파싱상태 - 사내 인포타임 {@code 구분코드2} 로 저장하는 코드값 (2026-09-23 신설).
+ *
+ * <p><b>⚠️ {@link ClassificationStatus}(분류상태)와 <b>다른 축</b>이다.
+ *
+ * 분류상태는 "기관분류를 확정했는가", 파싱상태는 "문서를 읽어 산출물을 만들었는가" 다.
+ *
+ * 파싱은 성공했는데 기관 테이블에 없는 기관이라 미분류로 남는 건이 정상적으로 존재한다.
+ *
+ * <p><b>⚠️ SQL 리터럴은 변환을 타지 않는다 - {@code WHERE 파싱상태구분 = '03'} 처럼 코드로 적을 것.
+ */
+public final class ParseStatus {
+
+    public static final String PENDING = "PENDING";
+
+    public static final String SUCCESS = "SUCCESS";
+
+    public static final String FAILED = "FAILED";
+
+    private static final Map<String, String> NAME_TO_CODE = new LinkedHashMap<String, String>();
+
+    private static final Map<String, String> CODE_TO_NAME = new LinkedHashMap<String, String>();
+
+    static {
+
+        put("01", PENDING);
+
+        put("02", SUCCESS);
+
+        put("03", FAILED);
+
+    }
+
+    private static void put(String code, String name) {
+
+        NAME_TO_CODE.put(name, code);
+
+        CODE_TO_NAME.put(code, name);
+
+    }
+
+    public static Map<String, String> nameToCode() {
+
+        return Collections.unmodifiableMap(NAME_TO_CODE);
+
+    }
+}
```

```

+   public static Map<String, String> codeToName() {
+       return Collections.unmodifiableMap(CODE_TO_NAME);
+   }
+
+   private ParseStatus() {}
+}

```

config/DocumentTypeHandler.java

TypeHandler (신규) CodeTypeHandler 상속. 빈 등록은 하지 않고 Mapper XML 에서 자리마다 FQCN 으로 쓴다.

```

new file mode 100644
@@ -0,0 +1,19 @@

+package com.kb.uploader.config;
+
+import com.kb.uploader.code.DocumentType;
+
+import java.util.Map;
+
+/** {@code BID_PROPOSAL} ↔ {@code '01'} - 자세한 이유는 {@link CodeTypeHandler}. */
+public class DocumentTypeHandler extends CodeTypeHandler {
+
+   @Override
+   protected Map<String, String> nameToCode() {
+       return DocumentType.nameToCode();
+   }
+
+   @Override
+   protected Map<String, String> codeToName() {
+       return DocumentType.codeToName();
+   }
+}

```

config/ParseStatusHandler.java

TypeHandler (신규)

```

new file mode 100644
@@ -0,0 +1,19 @@

+package com.kb.uploader.config;
+

```

```
+import com.kb.uploader.code.ParseStatus;
+
+import java.util.Map;
+
+/** {@code SUCCESS} ⇔ {@code '02'} - 자세한 이유는 {@link CodeTypeHandler}. */
+public class ParseStatusTypeHandler extends CodeTypeHandler {
+
+    @Override
+    protected Map<String, String> nameToCode() {
+        return ParseStatus.nameToCode();
+    }
+
+    @Override
+    protected Map<String, String> codeToName() {
+        return ParseStatus.codeToName();
+    }
+}
```

6. 맵퍼

```
mapper/UploadedFileMapper.java
```

매패 인터페이스 메서드 7개 추가. XML 의 id 와 이름이 정확히 같아야 한다.

[illegible]

```

+         @Param("offset") int offset,
+         @Param("limit") int limit);
+
+ long countRfps(@Param("institution") String institution,
+         @Param("docDate") String docDate);
+
+ }

```

mapper/UploadedFileMapper.xml

매퍼 XML — 가장 중요 resultMap 7열, 값목록 조각, 두 방언 insert, updateParseResult, 집계·검색 9문. resultMap 에 빠뜨리면 DB 에는 들어가는데 조회하면 늘 null 이다.

@@ -45,21 +45,70 @@

```

+         typeHandler="com.kb.uploader.config.LocalDateTimeStringTypeHandler"/>
+ <result property="classifiedAt"      column="분류일시"
+         typeHandler="com.kb.uploader.config.LocalDateTimeStringTypeHandler"/>
+ <result property="storedPath"        column="저장경로내용"/>
+ <!--
+
+   ⚠ 감사 2열은 **읽기에도 반드시 있어야 한다**(2026-09-09 추가). 종전에는 쓰기
+
+   (`#{...}`)만 있고 여기가 빠져 있어 DB 에는 값이 들어가는데 조회하면 늘
+
+   null 이었다 - 화면이 비어 보이는 게 첫 증상이다. 더 나쁜 건
+
+   KGI12100$FileStatusClassify 처럼 **findById 로 읽어 그대로 update 하는 경로**
+
+   에서 그 null 이 DB 로 되썬진다는 점이다.
+
+   -->
+
+   <result property="systemUsedAt"      column="시스템사용일시"
+
+         typeHandler="com.kb.uploader.config.LocalDateTimeStringTypeHandler"/>
+
+   <result property="systemUserNo"      column="시스템사용자번호"/>
+
+   <!--
+
+   — 2026-09-23 파싱 전환 7열 —
+
+   ⚠ 위 감사 2열과 똑같은 함정이 있다. 여기 빠뜨리면 DB 에는 값이 들어가는데 조회하면
+
+   늘 null 이고, 읽어서 그대로 update 하는 경로에서 그 null 이 DB 로 되썬진다.
+
+   -->
+
+   <result property="docType"           column="문서종류구분"
+
+         typeHandler="com.kb.uploader.config.DocumentTypeTypeHandler"/>
+
+   <result property="parseStatus"        column="파싱상태구분"
+
+         typeHandler="com.kb.uploader.config.ParseStatusTypeHandler"/>
+
+   <result property="parseMessage"        column="파싱메시지내용"/>
+
+   <result property="outputPath"         column="파싱산출물경로내용"/>
+
+   <result property="parsedAt"           column="파싱일시"

```

```

+         typeHandler="com.kb.uploader.config.LocalDateTimeStringTypeHandler"/>
+     <result property="pageCount"      column="문서페이지수"/>
+     <result property="docDate"       column="문서일자"/>
+
+ </resultMap>
+
+ <!-- 파라미터 쪽 TypeHandler 표기가 길어 sql 조각으로 묶는다 -->
+ <sql id="값목록">
+     #{institutionName},
+     - #{category,typeHandler=com.kb.uploader.config.InstitutionCategoryTypeHandler},
+
+     <include refid="기관분류값"/>,
+
+     #{year},
+     #{status,typeHandler=com.kb.uploader.config.ClassificationStatusTypeHandler},
+     #{originalName},
+     #{uploadedAt,typeHandler=com.kb.uploader.config.LocalDateTimeStringTypeHandler},
+     #{storedPath}
+ </sql>
+
+ <!--
+
+ + ⚠ 파싱 전환(2026-09-23) 이후 기관분류가 ***미분류*** 일 수 있다. 그 값은
+
+ + code/InstitutionCategory 의 코드표(01~05)에 없어서 TypeHandler 가 예외를 던진다.
+
+ + DDL 의 CHECK 제약도 코드 5종만 받는다. 그래서 **미분류는 NULL 로 저장**한다.
+
+ + 화면 표기는 UploadedFile.getCategoryLabel() 이 NULL 을 "미분류" 로 되돌린다.
+
+ -->
+
+ <sql id="기관분류값">
+
+     <choose>
+
+         <when test="category == null or category == '미분류'">NULL</when>
+
+         <otherwise>#{category,typeHandler=com.kb.uploader.config.InstitutionCategoryTypeHandler}</otherwise>
+
+     </choose>
+
+ </sql>
+
+
+ <!-- 파싱 7월의 값. insert 와 updateParseResult 가 같이 쓴다 -->
+
+ <sql id="파싱값목록">
+
+     #{docType,typeHandler=com.kb.uploader.config.DocumentTypeTypeHandler},
+
+     #{parseStatus,typeHandler=com.kb.uploader.config.ParseStatusTypeHandler},
+
+     #{parseMessage},
+
+     #{outputPath},
+
+     #{parsedAt,typeHandler=com.kb.uploader.config.LocalDateTimeStringTypeHandler},
+
+     #{pageCount},
+
+     #{docDate}
+
+ </sql>
+
+

```

<!--

⚠ 같은 id 가 두 번 나오는 자리는 **방언 분기**다(databaseId). 접속한 DB 에 맞는 쪽만 로드된다 - config/MyBatisConfig 가 값을 정한다.
한쪽만 고치면 다른 DB 에서 조용히 낡는다. **항상 둘을 같이 고칠 것.**

@@ -68,40 +117,180 @@

<!-- MySQL: useGeneratedKeys로 AUTO_INCREMENT id 채움 -->

<insert id="insert" databaseId="mysql" useGeneratedKeys="true" keyProperty="id">

INSERT INTO TSKGIAF01

(기관명, 기관영업분류구분, 문서년, 분류상태구분, 원본파일명, 업로드일시, 저장경로내용,

- 시스템사용일시, 시스템사용자번호)

+ 시스템사용일시, 시스템사용자번호,

+ 문서종류구분, 파싱상태구분, 파싱메시지내용, 파싱산출물경로내용, 파싱일시, 문서페이지수, 문서일자)

VALUES

(<include refid="값목록"/>,

- #{systemUsedAt,typeHandler=com.kb.uploader.config.LocalDateTimeStringTypeHandler}, #{systemUserNo})

+ #{systemUsedAt,typeHandler=com.kb.uploader.config.LocalDateTimeStringTypeHandler}, #{systemUserNo},

+ <include refid="파싱값목록"/>)

</insert>

<!-- Oracle: 시퀀스에서 먼저 ID를 받아 직접 넣는다 (schema-oracle.sql 의 SQ_KGI_AGENTFILETABLE) -->

<insert id="insert" databaseId="oracle">

<selectKey keyProperty="id" resultType="long" order="BEFORE">

SELECT SQ_KGI_AGENTFILETABLE.NEXTVAL FROM DUAL

</selectKey>

INSERT INTO TSKGIAF01

(업로드파일일련번호, 기관명, 기관영업분류구분, 문서년, 분류상태구분, 원본파일명, 업로드일시,

- 저장경로내용, 시스템사용일시, 시스템사용자번호)

+ 저장경로내용, 시스템사용일시, 시스템사용자번호,

+ 문서종류구분, 파싱상태구분, 파싱메시지내용, 파싱산출물경로내용, 파싱일시, 문서페이지수, 문서일자)

VALUES

(#{id}, <include refid="값목록"/>,

- #{systemUsedAt,typeHandler=com.kb.uploader.config.LocalDateTimeStringTypeHandler}, #{systemUserNo})

+ #{systemUsedAt,typeHandler=com.kb.uploader.config.LocalDateTimeStringTypeHandler}, #{systemUserNo},

+ <include refid="파싱값목록"/>)

</insert>

<update id="update">

UPDATE TSKGIAF01

SET

저장경로내용 = #{storedPath},

기관명 = #{institutionName},

문서년 = #{year},

- 기관영업분류구분 = #{category,typeHandler=com.kb.uploader.config.InstitutionCategoryTypeHandler},

+ 기관영업분류구분 = <include refid="기관분류값"/>,

분류상태구분 = #{status,typeHandler=com.kb.uploader.config.ClassificationStatusTypeHandler},

분류일시 = #{classifiedAt,typeHandler=com.kb.uploader.config.LocalDateTimeStringTypeHandler},

시스템사용일시 = #{systemUsedAt,typeHandler=com.kb.uploader.config.LocalDateTimeStringTypeHandler},

시스템사용자번호 = #{systemUserNo}

WHERE 업로드파일일련번호 = #{id}

</update>

+ <!--

+ 파싱 결과 기록 (2026-09-23 신설).

```

+  ⚠ JPA 였다면 더티체킹이 해 주던 UPDATE 다. 파싱 7열과 함께 **기관.분류상태도** 쓴다
+
+  - 파싱하면서 기관을 찾아 분류가 확정되기 때문이다.
+
+  -->
+
+  <update id="updateParseResult">
+
+    UPDATE TSKGIAF01
+
+    SET
+
+    문서종류구분 = #{docType,typeHandler=com.kb.uploader.config.DocumentTypeTypeHandler},
+
+    파싱상태구분 = #{parseStatus,typeHandler=com.kb.uploader.config.ParseStatusTypeHandler},
+
+    파싱메시지내용 = #{parseMessage},
+
+    파싱산출물경로내용 = #{outputPath},
+
+    파싱일시 = #{parsedAt,typeHandler=com.kb.uploader.config.LocalDateTimeStringTypeHandler},
+
+    문서페이지수 = #{pageCount},
+
+    문서일자 = #{docDate},
+
+    기관명 = #{institutionName},
+
+    기관영업분류구분 = <include refid="기관분류값"/>,
+
+    분류상태구분 = #{status,typeHandler=com.kb.uploader.config.ClassificationStatusTypeHandler},
+
+    분류일시 = #{classifiedAt,typeHandler=com.kb.uploader.config.LocalDateTimeStringTypeHandler},
+
+    시스템사용일시 = #{systemUsedAt,typeHandler=com.kb.uploader.config.LocalDateTimeStringTypeHandler},
+
+    시스템사용자번호 = #{systemUserNo}
+
+    WHERE 업로드파일일련번호 = #{id}
+
+  </update>
+
+
+  <!-- — 파싱 현황.대시보드 집계 (2026-09-23 신설) —————
+
+  ⚠ SQL 리터럴은 TypeHandler 를 타지 않는다. '01'.'02' 처럼 **코드**로 적을 것.
+
+  -->
+
+  <select id="countByParseStatus" resultType="long">
+
+    SELECT COUNT(*) FROM TSKGIAF01
+
+    WHERE 파싱상태구분 = #{parseStatus,typeHandler=com.kb.uploader.config.ParseStatusTypeHandler}
+
+  </select>
+
+
+  <select id="countByDocType" resultType="long">
+
+    SELECT COUNT(*) FROM TSKGIAF01
+
+    WHERE 문서종류구분 = #{docType,typeHandler=com.kb.uploader.config.DocumentTypeTypeHandler}
+
+  </select>
+
+

```

```

+ <!-- 파싱은 됐는데 기관을 못 찾은 건. 기관분류가 NULL 인 행이 그것이다 -->
+ <select id="countParsedUnclassified" resultType="long">
+     SELECT COUNT(*) FROM TSKGIAF01
+     WHERE 파싱상태구분 = '02' AND 기관영업분류구분 IS NULL
+ </select>
+
+ <select id="searchParseStatus" resultMap="uploadedFileMap">
+     SELECT * FROM TSKGIAF01
+     WHERE 분류상태구분 <= '04'
+     <if test="docType != null">
+         AND 문서종류구분 = #{docType,typeHandler=com.kb.uploader.config.DocumentTypeTypeHandler}
+     </if>
+     <if test="parseStatus != null">
+         AND 파싱상태구분 = #{parseStatus,typeHandler=com.kb.uploader.config.ParseStatusTypeHandler}
+     </if>
+     <if test="unclassifiedOnly != null and unclassifiedOnly">
+         AND 기관영업분류구분 IS NULL
+     </if>
+     ORDER BY 업로드파일일련번호 DESC
+ </select>
+
+ <!-- — 조회 API (2026-09-23 신설). 제안서 '01' / RFP '02', 파싱 성공 '02' 만 — -->
+ <sql id="제안서조건">
+     WHERE 문서종류구분 = '01' AND 파싱상태구분 = '02'
+     <if test="institution != null">
+         AND 기관명 LIKE '%' || #{institution} || '%'
+     </if>
+     <if test="docDate != null">
+         AND 문서일자 LIKE #{docDate} || '%'
+     </if>
+ </sql>
+
+ <sql id="제안서조건mysql">
+     WHERE 문서종류구분 = '01' AND 파싱상태구분 = '02'
+     <if test="institution != null">
+         AND 기관명 LIKE CONCAT('%', #{institution}, '%')

```

```

+   </if>
+   <if test="docDate != null">
+       AND 문서일자 LIKE CONCAT("#{docDate}", '%')
+   </if>
+ </sql>
+ <sql id="RFP조건">
+     WHERE 문서종류구분 = '02' AND 파싱상태구분 = '02'
+     <if test="institution != null">
+         AND 기관명 LIKE '%' || #{institution} || '%'
+     </if>
+     <if test="docDate != null">
+         AND 문서일자 LIKE #{docDate} || '%'
+     </if>
+ </sql>
+ <sql id="RFP조건mysql">
+     WHERE 문서종류구분 = '02' AND 파싱상태구분 = '02'
+     <if test="institution != null">
+         AND 기관명 LIKE CONCAT('%', #{institution}, '%')
+     </if>
+     <if test="docDate != null">
+         AND 문서일자 LIKE CONCAT("#{docDate}", '%')
+     </if>
+ </sql>
+
+ <select id="searchProposals" databaseId="mysql" resultMap="uploadedFileMap">
+     SELECT * FROM TSKGIAF01 <include refid="제안서조건mysql"/>
+     ORDER BY 업로드파일일련번호 DESC
+     LIMIT #{limit} OFFSET #{offset}
+ </select>
+ <select id="searchProposals" databaseId="oracle" resultMap="uploadedFileMap">
+     SELECT * FROM (
+         SELECT t.*, ROWNUM rnum FROM (
+             SELECT * FROM TSKGIAF01 <include refid="제안서조건"/>
+             ORDER BY 업로드파일일련번호 DESC
+         ) t WHERE ROWNUM <= #{offset} + #{limit}

```

```

+    ) WHERE rnum >= #{offset}

+ </select>

+ <select id="countProposals" databaseId="mysql" resultType="long">
+     SELECT COUNT(*) FROM TSKGIAF01 <include refid="제안서조건mysql"/>
+ </select>

+ <select id="countProposals" databaseId="oracle" resultType="long">
+     SELECT COUNT(*) FROM TSKGIAF01 <include refid="제안서조건"/>
+ </select>

+

+ <select id="searchRfps" databaseId="mysql" resultMap="uploadedFileMap">
+     SELECT * FROM TSKGIAF01 <include refid="RFP조건mysql"/>
+
+     ORDER BY 업로드파일일련번호 DESC
+
+     LIMIT #{limit} OFFSET #{offset}
+ </select>

+ <select id="searchRfps" databaseId="oracle" resultMap="uploadedFileMap">
+     SELECT * FROM (
+
+         SELECT t.*, ROWNUM rnum FROM (
+
+             SELECT * FROM TSKGIAF01 <include refid="RFP조건"/>
+
+             ORDER BY 업로드파일일련번호 DESC
+
+         ) t WHERE ROWNUM <= #{offset} + #{limit}
+
+     ) WHERE rnum >= #{offset}
+ </select>

+ <select id="countRfps" databaseId="mysql" resultType="long">
+     SELECT COUNT(*) FROM TSKGIAF01 <include refid="RFP조건mysql"/>
+ </select>

+ <select id="countRfps" databaseId="oracle" resultType="long">
+     SELECT COUNT(*) FROM TSKGIAF01 <include refid="RFP조건"/>
+ </select>

+

<!--
    ⚠ '04' = DELETED, '03' = REJECTED, '02' = CLASSIFIED (code/ClassificationStatus.java)
    ⚠ 이 둘은 파라미터에 시각 값이 없어 **컨트롤러가 LocalDateTime.now() 를 넘긴다.**
    DB 함수(TO_CHAR(SYSDATE)/DATE_FORMAT(NOW()))를 쓰면 방언이 갈리고 테스트로

```

7. 서비스

service/DocumentParseService.java

파싱 서비스 (신규) 파싱 → 기관매칭 → 산출물 생성 → updateParseResult. 감사 2열을 여기서 채운다.

```
new file mode 100644
@@ -0,0 +1,192 @@

+package com.kb.uploader.service;

+

+import com.fasterxml.jackson.databind.ObjectMapper;

+import com.fasterxml.jackson.databind.SerializationFeature;

+import com.kb.uploader.code.DocumentType;

+import com.kb.uploader.code.SystemUser;

+import com.kb.uploader.domain.Institution;

+import com.kb.uploader.domain.UploadedFile;

+import com.kb.uploader.mapper.InstitutionMapper;

+import com.kb.uploader.mapper.UploadedFileMapper;

+import com.kb.uploader.parse.DateDetector;

+import com.kb.uploader.parse.ExtractedDocument;

+import com.kb.uploader.parse.InstitutionMatcher;

+import com.kb.uploader.parse.PptExtractor;

+import com.kb.uploader.parse.ProposalDocument;

+import com.kb.uploader.parse.RfpExtractor;

+import com.kb.uploader.parse.RfpSummaryWriter;

+import com.kb.uploader.parse.SlideContent;

+import org.slf4j.Logger;

+import org.slf4j.LoggerFactory;

+import org.springframework.stereotype.Service;

+

+import java.nio.charset.StandardCharsets;

+import java.nio.file.Path;

+import java.nio.file.Paths;

+import java.time.LocalDateTime;

+import java.time.format.DateTimeFormatter;

+import java.util.ArrayList;
```

```

+import java.util.List;

+import java.util.Optional;

+

+/**
+ * 업로드 원본(userdata)을 파싱해 산출물을 만든다 (2026-09-23 신설).
+ *
+ * <ul>
+ * <li>입찰제안서: {@code {연도}_{기관분류}_{기관명}_{파싱yyyyMMdd}_bid_proposal.json}</li>
+ * <li>RFP: {@code {공고yyyyMMdd}_{기관분류}_{기관명}_{파싱yyyyMMdd}.md}</li>
+ * </ul>
+ *
+ * <p>충전의 {@code ClassificationService}(파일명으로 기관을 찾아 폴더를 옮기던 것)를 대신한다.
+ * 원본은 옮기지 않고 {@code userdata/} 에 그대로 둔다.
+ *
+ * <p>⚠ 감사 2열은 이 클래스가 채운다. 파싱은 배치·화면 양쪽에서 분리므로
+ * {@link SystemUser#get()} 값을 그대로 쓰고, 시각은 파싱일시와 같은 값으로 둔다.
+ */
+@Service
+public class DocumentParseService {
+
+    private static final Logger log = LoggerFactory.getLogger(DocumentParseService.class);
+
+    static final String UNKNOWN_NOTICE_DATE = "공고일미상";
+
+    private static final DateTimeFormatter DAY = DateTimeFormatter.ofPattern("yyyyMMdd");
+
+    private static final DateTimeFormatter STAMP = DateTimeFormatter.ofPattern("yyyy-MM-dd HH:mm:ss");
+
+    private final ObjectMapper mapper = new ObjectMapper().enable(SerializationFeature.INDENT_OUTPUT);
+
+    private final PptExtractor pptExtractor;
+
+    private final RfpExtractor rfpExtractor;
+
+    private final InstitutionMatcher institutionMatcher;
+
+    private final DateDetector dateDetector;
+
+    private final RfpSummaryWriter rfpSummaryWriter;
+
+    private final FileStorageService storageService;
+
+    private final InstitutionMapper institutionMapper;

```

```

+ private final UploadedFileMapper fileMapper;
+
+ public DocumentParseService(PptExtractor pptExtractor, RfpExtractor rfpExtractor,
+
+     InstitutionMatcher institutionMatcher, DateDetector dateDetector,
+
+     RfpSummaryWriter rfpSummaryWriter, FileStorageService storageService,
+
+     InstitutionMapper institutionMapper, UploadedFileMapper fileMapper) {
+
+     this.pptExtractor = pptExtractor;
+
+     this.rfpExtractor = rfpExtractor;
+
+     this.institutionMatcher = institutionMatcher;
+
+     this.dateDetector = dateDetector;
+
+     this.rfpSummaryWriter = rfpSummaryWriter;
+
+     this.storageService = storageService;
+
+     this.institutionMapper = institutionMapper;
+
+     this.fileMapper = fileMapper;
+
+ }
+
+ /** 파싱하고 결과(성공/실패)를 DB 에 기록한다. 예외를 밖으로 던지지 않는다. */
+ public UploadedFile parse(UploadedFile file) {
+
+     try {
+
+         if (DocumentType.BID_PROPOSAL.equals(file.getDocType())) {
+
+             parseProposal(file);
+
+         } else if (DocumentType.RFP.equals(file.getDocType())) {
+
+             parseRfp(file);
+
+         } else {
+
+             throw new IllegalStateException("문서 종류가 없는 이전 방식 업로드입니다");
+
+         }
+
+     } catch (Exception e) {
+
+         log.warn("파싱 실패: {}", file.getOriginalName(), e);
+
+         String msg = e.getMessage() != null ? e.getMessage() : e.getClass().getSimpleName();
+
+         file.markParseFailed("파싱 실패: " + msg);
+
+     }
+
+     stampAudit(file);
+
+     fileMapper.updateParseResult(file);
+
+     return file;
+
+ }

```

```

+
+  /** 이전 산출물을 지우고 userdata 원본으로 다시 파싱한다. */
+  public Optional<UploadedFile> reparse(Long id) {
+      Optional<UploadedFile> found = fileMapper.findById(id);
+      if (!found.isPresent()) return Optional.empty();
+      UploadedFile file = found.get();
+      try {
+          storageService.deleteIfExists(file.getOutputPath());
+      } catch (Exception e) {
+          log.warn("이전 산출물 삭제 실패: {}", file.getOutputPath(), e);
+      }
+      return Optional.of(parse(file));
+  }
+
+
+  private void parseProposal(UploadedFile file) throws Exception {
+      LocalDateTime now = LocalDateTime.now();
+      List<SlideContent> slides = pptExtractor.extract(Paths.get(file.getStoredPath()));
+      if (slides.isEmpty()) throw new IllegalStateException("슬라이드가 없습니다");
+
+      StringBuilder all = new StringBuilder();
+      for (SlideContent s : slides) {
+          all.append(s.getTitle()).append('\n').append(s.getTextFull()).append('\n');
+      }
+      InstitutionMatcher.Match inst = institutionMatcher.match(all.toString(), institutions());
+
+      List<String> notes = new ArrayList<String>();
+      Optional<String> detected = dateDetector.detectProposalYear(slides);
+      String year;
+      if (detected.isPresent()) {
+          year = detected.get();
+      } else {
+          notes.add("제안서 연도를 찾지 못해 업로드 연도로 대체");
+          year = String.valueOf(now.getYear());
+      }
+      note(notes, inst);

```

```

+
+     ProposalDocument doc = new ProposalDocument(file.getOriginalName(), now.format(STAMP), slides);
+
+     String fileName = String.join("_", year, inst.getCategory(), inst.getName(), now.format(DAY))
+
+         + "_bid_proposal.json";
+
+     Path out = storageService.writeOutput(storageService.getProposalJsonDir(), fileName,
+
+         mapper.writeValueAsBytes(doc));
+
+
+     file.markParsed(inst.getName(), inst.getCategory(), year, out.toString(), slides.size(), join(notes));
+
+ }
+
+
+ private void parseRfp(UploadedFile file) throws Exception {
+
+     LocalDateTime now = LocalDateTime.now();
+
+     ExtractedDocument doc = rfpExtractor.extract(Paths.get(file.getStoredPath()));
+
+     List<String> lines = doc.lines();
+
+
+
+     InstitutionMatcher.Match inst = institutionMatcher.match(doc.fullText(), institutions());
+
+     List<String> notes = new ArrayList<String>();
+
+     Optional<String> notice = dateDetector.detectNoticeDate(lines);
+
+     if (!notice.isPresent()) notes.add("공고일을 찾지 못함");
+
+     note(notes, inst);
+
+
+
+
+     String markdown = rfpSummaryWriter.write(doc, new RfpSummaryWriter.Meta(
+
+         file.getOriginalName(), inst.getName(), inst.getCategory(),
+
+         notice.orElse(null), now.format(STAMP)));
+
+
+
+     String fileName = String.join("_", notice.orElse(UNKNOWN_NOTICE_DATE),
+
+         inst.getCategory(), inst.getName(), now.format(DAY)) + ".md";
+
+     Path out = storageService.writeOutput(storageService.getRfpMdDir(), fileName,
+
+         markdown.getBytes(StandardCharsets.UTF_8));
+
+
+
+     file.markParsed(inst.getName(), inst.getCategory(), notice.orElse(null),
+
+         out.toString(), null, join(notes));
+
+ }
+
+
+
+ private List<Institution> institutions() {

```

```

+     return institutionMapper.findAll();
+ }
+
+ /**
+  * 감사 2열. 규칙(2026-09-08 확정)대로 <b>그 행을 마지막으로 건드린 시각</b>을 넣는다.
+  * 파싱은 분류일시가 없을 수도 있어 파싱일시를 쓴다.
+  */
+ private static void stampAudit(UploadedFile file) {
+     file.setSystemUserNo(SystemUser.get());
+     file.setSystemUsedAt(file.getParsedAt() != null ? file.getParsedAt() : LocalDateTime.now());
+ }
+
+ private static void note(List<String> notes, InstitutionMatcher.Match inst) {
+     if (InstitutionMatcher.UNKNOWN_NAME.equals(inst.getName())) {
+         notes.add("기관명을 찾지 못함");
+     } else if (!inst.isRegistered() && UploadedFile.UNKNOWN_CATEGORY.equals(inst.getCategory())) {
+         notes.add("기관 테이블에 없는 기관(" + inst.getName() + ") → 미분류");
+     }
+ }
+
+ private static String join(List<String> notes) {
+     return notes.isEmpty() ? null : String.join(" / ", notes);
+ }
+}

```

service/FileUploadService.java

업로드 서비스 파일명 해석을 버리고 확장자만 검증한 뒤 파싱을 부른다.

```

@@ -1,85 +1,78 @@
package com.kb.uploader.service;

+import com.kb.uploader.code.DocumentType;

import com.kb.uploader.code.SystemUser;
import com.kb.uploader.domain.UploadedFile;
-import com.kb.uploader.dto.ParsedFileName;

import com.kb.uploader.dto.UploadResultItem;
import com.kb.uploader.mapper.UploadedFileMapper;
+import org.slf4j.Logger;

+import org.slf4j.LoggerFactory;

```

```

import org.springframework.stereotype.Service;
import org.springframework.web.multipart.MultipartFile;

import java.nio.file.Path;
import java.util.ArrayList;
import java.util.List;
import java.util.Optional;

+/**
+ * 업로드 오케스트레이션 (2026-09-23 파싱 전환).
+ *
+ * <p>종전에는 파일명을 해석해 기관을 알아낸 뒤 폴더를 옮겼다. 이제는 <b>확장자만 검증</b>하고
+ * 원본을 userdata 에 보관한 다음 {@link DocumentParseService} 가 문서 내용을 읽는다.
+ *
+ * <p>⚠ 한 파일이 실패해도 나머지는 계속 처리한다 - 여러 건을 한 번에 올리는 화면이라
+ * 중간에 멈추면 무엇이 처리됐는지 알 수 없다.
+ */

@Service
public class FileUploadService {

- private final FileParserService parserService;

+ private static final Logger log = LoggerFactory.getLogger(FileUploadService.class);

+

private final FileStorageService storageService;
- private final ClassificationService classificationService;

+ private final DocumentParseService parseService;

private final UploadedFileMapper fileMapper;

- public FileUploadService(FileParserService parserService,
-
- FileStorageService storageService,
-
- ClassificationService classificationService,
+ public FileUploadService(FileStorageService storageService,
+
+ DocumentParseService parseService,
+
+ UploadedFileMapper fileMapper) {
- this.parserService = parserService;

this.storageService = storageService;
- this.classificationService = classificationService;

+ this.parseService = parseService;

this.fileMapper = fileMapper;
}

- public List<UploadResultItem> upload(List<MultipartFile> files) {

- return upload(files, "", "", "");

- }

```

```

-
- public List<UploadResultItem> upload(List<MultipartFile> files,
-                                     String instOverride, String yearOverride, String categoryOverride) {
-     List<UploadResultItem> results = new ArrayList<>();
+ /**
+  * @param docType {@link DocumentType#BID_PROPOSAL} 또는 {@link DocumentType#RFP}
+  */
+ public List<UploadResultItem> upload(String docType, List<MultipartFile> files) {
+     List<UploadResultItem> results = new ArrayList<UploadResultItem>();
+     for (MultipartFile file : files) {
-         String originalName = file.getOriginalFilename();
+         String originalName = FileStorageService.safeFileName(file.getOriginalFilename());
+         if (file.isEmpty()
+             && (file.getOriginalFilename() == null || file.getOriginalFilename().isEmpty())) {
+             continue; // 파일을 고르지 않고 전송한 빈 파트
+         }
+         if (!DocumentType.accepts(docType, originalName)) {
+             results.add(new UploadResultItem(originalName, false, null,
+                 "허용되지 않는 확장자 ("
+                 + String.join(", ", DocumentType.extensions(docType)) + "만 가능"));
+             continue;
+         }
+         try {
-             Optional<ParsedFileName> parsed = parserService.parse(originalName);
-             if (!parsed.isPresent()) {
-                 boolean hasOverride = instOverride != null && !instOverride.trim().isEmpty();
-                 Path saved = storageService.saveToUnclassified(file, originalName);
-                 UploadedFile entity = new UploadedFile(
-                     originalName, saved.toString(),
-                     (yearOverride != null && !yearOverride.trim().isEmpty()) ? yearOverride.trim() : null,
-                     hasOverride ? instOverride.trim() : "알수없음");
-                 entity.setSystemUserNo(SystemUser.get());
-                 entity.setSystemUsedAt(entity.getUploadedAt());
-                 fileMapper.insert(entity);
-                 if (hasOverride) {
-                     boolean ok = classificationService.classify(entity);

```

```

-         results.add(new UploadResultItem(originalName, ok,
-
-             ok ? entity.getCategory() : null,
-
-             ok ? "수동 입력으로 분류 완료" : "미분류 (기관 미등록 - 기관 관리에서 등록 필요)"));
-
-     } else {
-
-         results.add(new UploadResultItem(originalName, false, null,
-
-             "파일명 형식 불일치 (년도_기관명_설명.확장자 필요)"));
-
-     }
-
-     continue;
-
- }
-
-     ParsedFileName p = parsed.get();
-
-     Path saved = storageService.saveToUnclassified(file, originalName);
-
-     UploadedFile entity = new UploadedFile(
-
-         originalName, saved.toString(), p.getYear(), p.getInstitutionName());
+
-     Path saved = storageService.saveOriginal(file, originalName);
+
-     UploadedFile entity = new UploadedFile(docType, originalName, saved.toString());
-
-     entity.setSystemUserNo(SystemUser.get());
-     entity.setSystemUsedAt(entity.getUploadedAt());
-
-     fileMapper.insert(entity);
+
-     entity.setSystemUsedAt(entity.getUploadedAt());
+
-     fileMapper.insert(entity);
-
-     boolean ok = classificationService.classify(entity);
-
-     results.add(new UploadResultItem(
-
-         originalName, ok,
-
-         ok ? entity.getCategory() : null,
-
-         ok ? "분류 완료" : "미분류 (기관 미등록)"));
+
-     UploadedFile parsed = parseService.parse(entity);
+
-     boolean ok = "SUCCESS".equals(parsed.getParseStatus());
+
-     results.add(new UploadResultItem(originalName, ok,
+
-         ok ? parsed.getCategoryLabel() : null,
+
-         ok ? "파싱 완료 → " + parsed.getOutputFileName() : parsed.getParseMessage()));
-
-     } catch (Exception e) {
-         results.add(new UploadResultItem(originalName, false, null,
-
-             "오류: " + e.getMessage()));
+
-         log.error("업로드 처리 실패: {}", originalName, e);
+
-         results.add(new UploadResultItem(originalName, false, null, "오류: " + e.getMessage()));

```

```

    }
}
return results;
}

```

service/FileStorageService.java

저장 서비스 원본 보관(userdata)과 산출물 쓰기를 더한다. cleanEmptyClassifiedDirs 는 KGI12500 이 쓰므로 남긴다.

```

@@ -5,8 +5,9 @@ import org.slf4j.LoggerFactory;

import org.springframework.beans.factory.annotation.Value;
import org.springframework.stereotype.Service;
import org.springframework.web.multipart.MultipartFile;
import java.io.IOException;
+import java.io.InputStream;

import java.nio.file.*;
import java.nio.file.attribute.BasicFileAttributes;
import java.time.LocalDateTime;
import java.time.format.DateTimeFormatter;
@@ -18,11 +19,21 @@ public class FileStorageService {

    private static final DateTimeFormatter TS_FMT =
        DateTimeFormatter.ofPattern("yyyyMMddHHmmss");

    private final String baseDir;
+    // — 2026-09-23 파싱 전환: 원본 보관 / 산출물 경로 —
+
+    private final String userdataDir;
+
+    private final String proposalJsonDir;
+
+    private final String rfpMdDir;

-    public FileStorageService(@Value("${upload.base-dir}") String baseDir) {
+    public FileStorageService(@Value("${upload.base-dir}") String baseDir,
+
+        @Value("${upload.userdata-dir}") String userdataDir,
+
+        @Value("${upload.proposal-json-dir}") String proposalJsonDir,
+
+        @Value("${upload.rfp-md-dir}") String rfpMdDir) {

        this.baseDir = baseDir;
+        this.userdataDir = userdataDir;
+
+        this.proposalJsonDir = proposalJsonDir;
+
+        this.rfpMdDir = rfpMdDir;

    }

    public Path saveToUnclassified(MultipartFile file, String filename) throws IOException {
        Path dir = Paths.get(baseDir, "unclassified");
@@ -35,9 +46,14 @@ public class FileStorageService {

        ? filename.substring(0, dot) + "_" + ts + filename.substring(dot)
        : filename + "_" + ts;
        target = dir.resolve(filename);
    }
-    file.transferTo(target.toFile());
+
+    // transferTo(File)은 내부적으로 Part.write(경로)를 호출하는데, 서블릿 스펙상 그 경로는

```

```

+ // multipart 임시 위치 기준 상대경로다. WebLogic은 절대경로를 그대로 이어 붙여
+ // "<도메인>/.../C:/uploader-local/..." 같은 잘못된 경로를 만든다. 스트림 복사로 우회한다.
+ try (InputStream in = file.getInputStream()) {
+     Files.copy(in, target, StandardCopyOption.REPLACE_EXISTING);
+ }
+
+     return target;
+ }
+
+     public Path moveToClassified(Path source, String category,
@@ -93,5 +109,68 @@ public class FileStorageService {
+
+     public Path buildUnclassifiedPath(String filename) {
+         return Paths.get(baseDir, "unclassified", filename);
+     }
+
+
+ // — 2026-09-23 파싱 전환 —————
+ // 원본을 옮기지 않고 userdata 에 보관하고, 파싱 산출물을 따로 쓴다.
+
+ /** 원본 보관. 같은 이름이 있으면 _yyyyMMddHHmmss 를 붙인다. */
+ public Path saveOriginal(MultipartFile file, String filename) throws IOException {
+     Path dir = Paths.get(userdataDir);
+     Files.createDirectories(dir);
+     filename = safeFileName(filename);
+     Path target = dir.resolve(filename);
+
+     if (Files.exists(target)) {
+         int dot = filename.lastIndexOf('.');
+         String ts = LocalDateTime.now().format(TS_FMT);
+         filename = (dot >= 0)
+             ? filename.substring(0, dot) + "_" + ts + filename.substring(dot)
+             : filename + "_" + ts;
+         target = dir.resolve(filename);
+     }
+
+     // ⚠ transferTo(File) 을 쓰지 않는 이유는 saveToUnclassified 주석 참조(WebLogic 경로 문제).
+
+     try (InputStream in = file.getInputStream()) {
+         Files.copy(in, target, StandardCopyOption.REPLACE_EXISTING);
+     }
+
+     return target;
+ }
+
+ }
+
+

```

```

+  /** 산출물(JSON·MD)을 쓴다. 같은 이름이 있으면 _2, _3 ... 을 붙인다. */
+  public Path writeOutput(Path dir, String filename, byte[] content) throws IOException {
+      Files.createDirectories(dir);
+      filename = safeFileName(filename);
+      Path target = dir.resolve(filename);
+      int dot = filename.lastIndexOf('.');
+      String base = dot >= 0 ? filename.substring(0, dot) : filename;
+      String ext = dot >= 0 ? filename.substring(dot) : "";
+      for (int n = 2; Files.exists(target); n++) {
+          target = dir.resolve(base + "_" + n + ext);
+      }
+      Files.write(target, content);
+      return target;
+  }
+
+  /** 재파싱 때 이전 산출물을 지운다. 없으면 조용히 넘어간다. */
+  public void deleteIfExists(String path) throws IOException {
+      if (path == null || path.trim().isEmpty()) return;
+      Files.deleteIfExists(Paths.get(path));
+  }
+
+  /**
+   * 파일명에서 경로 구분자를 떼어 낸다.
+   * ⚠ 브라우저에 따라 전체 경로를 보내고, 악의적 요청은 "../" 를 섞는다.
+   * 그대로 resolve 하면 저장 디렉터리 밖으로 나간다.
+   */
+  public static String safeFileName(String name) {
+      if (name == null) return "unnamed";
+      String only = name.replace('\\', '/');
+      int slash = only.lastIndexOf('/');
+      if (slash >= 0) only = only.substring(slash + 1);
+      only = only.trim();
+      return only.isEmpty() ? "unnamed" : only;
+  }
+
+

```

```

+ public Path getUserdataDir() { return Paths.get(userdataDir); }
+ public Path getProposalJsonDir() { return Paths.get(proposalJsonDir); }
+ public Path getRfpMdDir() { return Paths.get(rfpMdDir); }

}

```

service/InstitutionService.java

기관 서비스 기관 저장 후 자동 재분류를 없앤다. 재파싱은 화면에서 사람이 누른다.

```

@@ -21,16 +21,13 @@ public class InstitutionService {

    private static final ObjectMapper MAPPER = new ObjectMapper();

    private final InstitutionMapper institutionMapper;
- private final ClassificationService classificationService;

    private final UploadedFileMapper uploadedFileMapper;

    public InstitutionService(InstitutionMapper institutionMapper,
- ClassificationService classificationService,

        UploadedFileMapper uploadedFileMapper) {
        this.institutionMapper = institutionMapper;
- this.classificationService = classificationService;

        this.uploadedFileMapper = uploadedFileMapper;
    }

    public List<Institution> findAll() {
@@ -73,10 +70,10 @@ public class InstitutionService {

        Institution created = new Institution(name, category);
        created.setSystemUserNo(SystemUser.get());
        institutionMapper.insert(created);
    });
- List<UploadedFile> pending = uploadedFileMapper.findByStatus("UNCLASSIFIED");
- pending.forEach(f -> classificationService.classify(f));

+ // 2026-09-23: 자동 재분류를 없앴다. 기관을 고친 뒤 다시 돌리려면
+ // 파싱 현황 화면의 [재파싱] 을 누른다.

    }

    public byte[] exportToXlsx() throws IOException {
        try (Workbook wb = new XSSFWorkbook()) {
@@ -116,8 +113,8 @@ public class InstitutionService {

        institutionMapper.insert(created);
    }
    }

- List<UploadedFile> pending = uploadedFileMapper.findByStatus("UNCLASSIFIED");
- pending.forEach(f -> classificationService.classify(f));

+ // 2026-09-23: 자동 재분류를 없앴다. 기관을 고친 뒤 다시 돌리려면
+ // 파싱 현황 화면의 [재파싱] 을 누른다.

```

}
}

8. 컨트롤러

controller/KGI10000\$DashboardView.java

대시보드 KPI 축을 분류 → 파싱으로 바꾼다.

```
@@ -25,18 +25,22 @@ public class KGI10000$DashboardView {  
  
    @RequestParam(required = false) String keyword,  
    @RequestParam(required = false, defaultValue = "1") int page,  
    Model model) {  
  
+    // — 2026-09-23 파싱 전환: KPI 축이 분류 → 파싱으로 바뀌었다 —  
  
        long total      = fileMapper.countAll();  
-        long classified = fileMapper.countByStatus("CLASSIFIED");  
  
-        long unclassified = fileMapper.countByStatus("UNCLASSIFIED");  
  
-        long excluded    = fileMapper.countByStatus("REJECTED") + fileMapper.countByStatus("DELETED");  
  
+        long parseSuccess = fileMapper.countByParseStatus("SUCCESS");  
+        long parseFailed  = fileMapper.countByParseStatus("FAILED");  
+        long unclassified = fileMapper.countParsedUnclassified();  
  
        model.addAttribute("total", total);  
-        model.addAttribute("classified", classified);  
  
+        model.addAttribute("parseSuccess", parseSuccess);  
+        model.addAttribute("parseFailed", parseFailed);  
  
        model.addAttribute("unclassified", unclassified);  
-        model.addAttribute("excluded", excluded);  
  
-        model.addAttribute("unclassifiedCount", unclassified);  
  
+        model.addAttribute("proposalCount", fileMapper.countByDocType("BID_PROPOSAL"));  
+        model.addAttribute("rfpCount", fileMapper.countByDocType("RFP"));  
  
+        // 좌측 네비 배지 - 미분류 건수에서 파싱 실패 건수로 바뀌었다  
  
+        model.addAttribute("parseFailedCount", parseFailed);  
  
  
        List<UploadedFile> recent = fileMapper.findRecent(10);  
        model.addAttribute("recentFiles", recent);  
    }  
}
```

controller/KGI11100\$UploadAction.java

업로드 실행 창구를 제안서/RFP 둘로 나눈다. 기관·연도 수동 입력 파라미터를 없앤다.

```
@@ -1,8 +1,8 @@  
  
package com.kb.uploader.controller;  
  
+import com.kb.uploader.code.DocumentType;  
  
import com.kb.uploader.dto.UploadResultItem;
```

```

-import com.kb.uploader.mapper.InstitutionMapper;

import com.kb.uploader.mapper.UploadedFileMapper;
import com.kb.uploader.service.FileUploadService;
import org.springframework.stereotype.Controller;
import org.springframework.ui.Model;
@@ -13,32 +13,41 @@ import org.springframework.web.multipart.MultipartFile;

import java.util.List;
import java.util.stream.Collectors;

+/**
+ * 업로드 실행 (2026-09-23 파싱 전환).
+ *
+ * <p>업로드 창구가 <b>둘</b>로 갈렸다. 문서 종류마다 받는 확장자가 다르고 만드는 산출물도 다르다.
+ * 기관·연도·분류를 손으로 넣던 파라미터는 없앴다 - 이제 문서 본문에서 찾는다.
+ */

@Controller
@RequestMapping("/upload")
public class KGI11100$UploadAction {

    private final FileUploadService uploadService;
    private final UploadedFileMapper fileMapper;
-    private final InstitutionMapper instMapper;

    public KGI11100$UploadAction(FileUploadService uploadService,
-        UploadedFileMapper fileMapper,
-        InstitutionMapper instMapper) {
+        UploadedFileMapper fileMapper) {

        this.uploadService = uploadService;
        this.fileMapper = fileMapper;
-        this.instMapper = instMapper;

    }

-    @PostMapping
-    public String execute(@RequestParam("files") List<MultipartFile> files,
-        @RequestParam(defaultValue = "") String institution,
-        @RequestParam(defaultValue = "") String year,
-        @RequestParam(defaultValue = "") String category,
-        Model model) {
-        populateCategories(model);
-        model.addAttribute("unclassifiedCount", fileMapper.countByStatus("UNCLASSIFIED"));
+    /** 입찰제안서 (ppt, pptx) */
+    @PostMapping("/proposal")
+    public String executeProposal(@RequestParam("files") List<MultipartFile> files, Model model) {
+        return handle(DocumentType.BID_PROPOSAL, files, model);

```

```

+   }
+
+   /** 임찰공고문 RFP (pdf, hwp, hwpX) */
+   @PostMapping("/rfp")
+   public String executeRfp(@RequestParam("files") List<MultipartFile> files, Model model) {
+       return handle(DocumentType.RFP, files, model);
+   }
+
+
+   private String handle(String docType, List<MultipartFile> files, Model model) {
+       model.addAttribute("parseFailedCount", fileMapper.countByParseStatus("FAILED"));
+
+       List<MultipartFile> validFiles = files.stream()
+           .filter(f -> !f.isEmpty())
+           .collect(Collectors.toList());
+@@ -46,18 +55,10 @@ public class KGI11100$UploadAction {
+
+       model.addAttribute("errorMessage", "파일을 선택해 주세요.");
+       return "upload";
+   }
+
+   List<UploadResultItem> results = uploadService.upload(validFiles, institution, year, category);
+
+   List<UploadResultItem> results = uploadService.upload(docType, validFiles);
+
+   model.addAttribute("results", results);
+   model.addAttribute("uploadedType", DocumentType.label(docType));
+
+   return "upload";
+   }
+
+   private void populateCategories(Model model) {
+
+       List<String> categories = instMapper.findAll().stream()
+
+           .map(i -> i.getCategory())
+
+           .filter(c -> c != null && !c.trim().isEmpty())
+
+           .distinct().sorted()
+
+           .collect(Collectors.toList());
+
+       model.addAttribute("categories", categories);
+
+   }
+
+   -}
+
+ \ No newline at end of file
+}

```

controller/KGI12000\$ParseStatusList.java

파싱 현황 (신규) 옛 KGI12000\$FileStatusList 를 대신한다. 그 파일은 지운다.

new file mode 100644

@@ -0,0 +1,54 @@

```
+package com.kb.uploader.controller;

+

+import com.kb.uploader.code.DocumentType;

+import com.kb.uploader.domain.UploadedFile;

+import com.kb.uploader.mapper.UploadedFileMapper;

+import org.springframework.stereotype.Controller;

+import org.springframework.ui.Model;

+import org.springframework.util.StringUtils;

+import org.springframework.web.bind.annotation.GetMapping;

+import org.springframework.web.bind.annotation.RequestMapping;

+import org.springframework.web.bind.annotation.RequestParam;

+

+import java.util.List;

+

+/**
+ * 파싱 현황 목록 (2026-09-23 신설, 옛 "파일 처리 상태" 화면을 대신한다).
+ *
+ * <p>분류 축 하나로 보던 화면을 <b>파싱 축</b>으로 바꾼 것이다. 파싱 성공/실패와
+ * 기관 미분류가 서로 다른 축이라 필터도 따로 둔다.
+ */
+@Controller
+@RequestMapping("/parse-status")
+public class KGI12000$ParseStatusList {

+

+    private final UploadedFileMapper fileMapper;

+

+    public KGI12000$ParseStatusList(UploadedFileMapper fileMapper) {

+        this.fileMapper = fileMapper;

+    }

+

+    @GetMapping

+    public String execute(@RequestParam(required = false) String docType,

+                          @RequestParam(required = false) String parseStatus,

+                          @RequestParam(required = false) String category,
```

```

+         Model model) {
+
+         boolean unclassifiedOnly = "미분류".equals(category);
+
+         List<UploadedFile> files = fileMapper.searchParseStatus(
+
+             emptyOrNull(docType), emptyOrNull(parseStatus), unclassifiedOnly);
+
+
+         model.addAttribute("files", files);
+
+         model.addAttribute("docTypes", DocumentType.nameToCode().keySet());
+
+         model.addAttribute("docType", docType);
+
+         model.addAttribute("parseStatus", parseStatus);
+
+         model.addAttribute("category", category);
+
+         model.addAttribute("successCount", fileMapper.countByParseStatus("SUCCESS"));
+
+         model.addAttribute("parseFailedCount", fileMapper.countByParseStatus("FAILED"));
+
+         model.addAttribute("unclassifiedCount", fileMapper.countParsedUnclassified());
+
+         return "parse-status";
+
+     }
+
+
+     private static String emptyOrNull(String s) {
+
+         return StringUtils.hasText(s) ? s : null;
+
+     }
+
+ }

```

controller/KGI12110\$ParseReparse.java

재파싱 (신규) 5분 주기 배치를 대신하는 수동 버튼.

```

new file mode 100644
@@ -0,0 +1,43 @@
+package com.kb.uploader.controller;
+
+
+import com.kb.uploader.domain.UploadedFile;
+import com.kb.uploader.service.DocumentParseService;
+import org.springframework.stereotype.Controller;
+import org.springframework.web.bind.annotation.PathVariable;
+import org.springframework.web.bind.annotation.PostMapping;
+import org.springframework.web.bind.annotation.RequestMapping;
+import org.springframework.web.servlet.mvc.support.RedirectAttributes;
+
+
+import java.util.Optional;

```

```

+
+/**
+ * 수동 재파싱 (2026-09-23 신설).
+ *
+ * <p>종전의 5분 주기 재분류 배치를 대신한다. 기관 정보를 고친 뒤 다시 돌리고 싶을 때 누른다.
+ * 감사 컬럼의 시스템사용자번호는 SystemUserInterceptor 가 이 클래스명에서 뽑아 넣는다.
+ */
+@Controller
+@RequestMapping("/parse-status")
+public class KGI12110$ParseReparse {
+
+    private final DocumentParseService parseService;
+
+    public KGI12110$ParseReparse(DocumentParseService parseService) {
+        this.parseService = parseService;
+    }
+
+    @PostMapping("/{id}/reparse")
+    public String execute(@PathVariable Long id, RedirectAttributes ra) {
+        Optional<UploadedFile> result = parseService.reparse(id);
+        if (!result.isPresent()) {
+            ra.addFlashAttribute("error", "파일을 찾을 수 없습니다 (id=" + id + ")");
+        } else if ("SUCCESS".equals(result.get().getParseStatus())) {
+            ra.addFlashAttribute("message", "재파싱 완료: " + result.get().getOriginalName()
+                + " → " + result.get().getOutputFileName());
+        } else {
+            ra.addFlashAttribute("error", "재파싱 실패: " + result.get().getOriginalName()
+                + " - " + result.get().getParseMessage());
+        }
+        return "redirect:/parse-status";
+    }
+}

```

controller/KGI12120\$ParseOutput.java

산출물 보기 (신규) Content-Type 을 text/plain; charset=UTF-8 로 고정한다. 안 그러면 한글이 깨진다.

new file mode 100644

@@ -0,0 +1,56 @@

```
+package com.kb.uploader.controller;
```

```
+
```

```
+import com.kb.uploader.domain.UploadedFile;
```

```
+import com.kb.uploader.mapper.UploadedFileMapper;
```

```
+import org.springframework.http.ContentDisposition;
```

```
+import org.springframework.http.HttpHeaders;
```

```
+import org.springframework.http.HttpStatus;
```

```
+import org.springframework.http.MediaType;
```

```
+import org.springframework.http.ResponseEntity;
```

```
+import org.springframework.stereotype.Controller;
```

```
+import org.springframework.web.bind.annotation.*;
```

```
+
```

```
+import java.io.IOException;
```

```
+import java.nio.charset.StandardCharsets;
```

```
+import java.nio.file.Files;
```

```
+import java.nio.file.Path;
```

```
+import java.nio.file.Paths;
```

```
+import java.util.Optional;
```

```
+
```

```
+/**
```

```
+ * 파싱 산출물(JSON/MD) 보기·내려받기 (2026-09-23 신설).
```

```
+ *
```

```
+ * <p>⚠ Content-Type 을 text/plain; charset=UTF-8 로 고정한다. 브라우저가 인코딩을 추측하면
```

```
+ * 한글 요약이 깨져 보인다.
```

```
+ */
```

```
+@Controller
```

```
+@RequestMapping("/parse-status")
```

```
+public class KGI12120$ParseOutput {
```

```
+
```

```
+    private final UploadedFileMapper fileMapper;
```

```
+
```

```
+    public KGI12120$ParseOutput(UploadedFileMapper fileMapper) {
```

```
+        this.fileMapper = fileMapper;
```

```

+    }
+
+    @GetMapping("/{id}/output")
+    @ResponseBody
+    public ResponseEntity<byte[]> execute(@PathVariable Long id,
+
+                                           @RequestParam(defaultValue = "false") boolean download)
+        throws IOException {
+        Optional<UploadedFile> file = fileMapper.findById(id);
+        if (!file.isPresent() || file.get().getOutputPath() == null) {
+            return ResponseEntity.status(HttpStatus.NOT_FOUND).build();
+        }
+        Path path = Paths.get(file.get().getOutputPath());
+        if (!Files.exists(path)) {
+            return ResponseEntity.status(HttpStatus.NOT_FOUND).build();
+        }
+        ContentDisposition disposition = ContentDisposition.builder(download ? "attachment" : "inline")
+            .filename(path.getFileName().toString(), StandardCharsets.UTF_8).build();
+        return ResponseEntity.ok()
+            .header(HttpHeaders.CONTENT_DISPOSITION, disposition.toString())
+            .contentType(new MediaType("text", "plain", StandardCharsets.UTF_8))
+            .body(Files.readAllBytes(path));
+    }
+}

```

controller/KGI12100\$FileStatusClassify.java

기관 지정 재파싱 폴더 이동이 사라졌으므로, 기관을 등록하고 곧바로 재파싱하는 일로 바꾼다.

```

@@ -1,10 +1,10 @@
package com.kb.uploader.controller;

-import java.time.LocalDateTime;
-
-import com.kb.uploader.code.SystemUser;
+import com.kb.uploader.domain.UploadedFile;
+
+import com.kb.uploader.mapper.UploadedFileMapper;
+import com.kb.uploader.service.FileStorageService;
+import com.kb.uploader.service.DocumentParseService;
+import com.kb.uploader.service.InstitutionService;
+
import org.slf4j.Logger;

```

```

import org.slf4j.LoggerFactory;
import org.springframework.stereotype.Controller;
import org.springframework.web.bind.annotation.PathVariable;
@@ -12,54 +12,68 @@ import org.springframework.web.bind.annotation.PostMapping;

import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.RequestParam;
import org.springframework.web.servlet.mvc.support.RedirectAttributes;

+import java.util.Optional;
+
+/**
+ * 기관을 지정해 다시 분류한다 (2026-09-23 파싱 전환에 맞춰 동작 교체).
+ *
+ * <p>종전에는 파일을 {@code classified/} 폴더로 <b>옮기는</b> 것이 분류였다. 이제는 폴더를 옮기지
+ * 않으므로 이 화면의 일은 <b>기관을 기관 테이블에 등록하고 그 문서를 재파싱</b>하는 것이다.
+ * 재파싱하면 InstitutionMatcher 가 방금 등록한 기관을 찾아 기관분류가 확정되고,
+ * 산출물 파일명도 {@code {날짜}_{기관분류}_{기관명}_...} 으로 다시 만들어진다.
+ *
+ * <p>⚠ 기관명만 바꾸고 재파싱을 하지 않으면 산출물 파일명이 옛 기관명 그대로 남아
+ * DB 와 파일이 어긋난다. 그래서 두 가지를 한 번에 한다.
+ */
@Controller
@RequestMapping("/file-status")
public class KGI12100$FileStatusClassify {

    private static final Logger log = LoggerFactory.getLogger(KGI12100$FileStatusClassify.class);

    private final UploadedFileMapper fileMapper;
    - private final FileStorageService storageService;
    + private final InstitutionService institutionService;
    + private final DocumentParseService parseService;

    public KGI12100$FileStatusClassify(UploadedFileMapper fileMapper,
    - FileStorageService storageService) {
    + InstitutionService institutionService,
    + DocumentParseService parseService) {

        this.fileMapper = fileMapper;
    - this.storageService = storageService;
    + this.institutionService = institutionService;
    + this.parseService = parseService;

    }

    @PostMapping("/{id}/classify")
    public String execute(@PathVariable Long id,
                        @RequestParam String category,
                        @RequestParam(defaultValue = "") String institution,

```

```

-         @RequestParam(defaultValue = "") String year,
-         RedirectAttributes ra) {
-     fileMapper.findById(id).ifPresent(file -> {
-
-         if (!institution.trim().isEmpty()) {
-
-             file.setInstitutionName(institution.trim());
-
-         }
-
-         if (!year.trim().isEmpty()) {
-
-             file.setYear(year.trim());
-
-         }
-
-         String instName = file.getInstitutionName() != null ? file.getInstitutionName() : "알수없음";
-
-         try {
-
-             java.nio.file.Path source = java.nio.file.Paths.get(file.getStoredPath());
-
-             java.nio.file.Path dest = storageService.moveToClassified(
-
-                 source,
-
-                 category,
-
-                 file.getYear() != null ? file.getYear() : "미확인",
-
-                 instName
-
-             );
-
-             file.classify(category, dest.toString());
-
-             file.setSystemUserNo(SystemUser.get());
-
-             file.setSystemUsedAt(
-
-                 file.getClassifiedAt() != null ? file.getClassifiedAt() : LocalDateTime.now());
-
-             fileMapper.update(file);
-
-         } catch (Exception e) {
-
-             log.warn("수동 분류 처리 실패: {}", file.getOriginalName(), e);
-
-         }
-
-         Optional<UploadedFile> found = fileMapper.findById(id);
-
-         if (!found.isPresent()) {
-
-             ra.addFlashAttribute("error", "파일을 찾을 수 없습니다 (id=" + id + ")");
-
-             return "redirect:/file-status/classified";
-
-         }
-
-         String name = institution.trim();
-
-         if (name.isEmpty()) {
-
-             ra.addFlashAttribute("error", "기관명을 입력해 주세요.");
-
-             return "redirect:/file-status/classified";
-
-         }
-
-     }
-
- }

```

```

+     try {
+         institutionService.save(name, category.trim());
+         Optional<UploadedFile> result = parseService.reparse(id);
+         if (result.isPresent() && "SUCCESS".equals(result.get().getParseStatus())) {
+             ra.addFlashAttribute("message",
+
+                 "기관 등록 후 재파싱 완료: " + name + " → " + result.get().getOutputFileName());
+
+         } else {
+             ra.addFlashAttribute("error", "기관은 등록했으나 재파싱에 실패했습니다"
+
+                 + (result.isPresent() ? " - " + result.get().getParseMessage() : ""));
+
+         }
-     });
-     ra.addFlashAttribute("message", "분류 처리 완료");
-     return "redirect:/file-status";
+     } catch (Exception e) {
+         log.warn("기관 지정 재파싱 실패: id={}", id, e);
+         ra.addFlashAttribute("error", "처리 실패: " + e.getMessage());
+     }
+     return "redirect:/file-status/classified";
+
+ }
}

```

controller/KGI12400\$ClassifiedFileList.java

기관 미분류 목록 조건을 '파싱 성공 + 기관분류 NULL' 로 바꾼다. 옛 조건(기관명='알수없음')은 새 경로에서 맞지 않는다.

```

@@ -20,14 +20,21 @@ public class KGI12400$ClassifiedFileList {
    public KGI12400$ClassifiedFileList(UploadedFileMapper fileMapper) {
        this.fileMapper = fileMapper;
    }
+
+    /**
+     * 파싱은 됐는데 <b>기관을 못 찾은</b> 건을 모은다 (2026-09-23 파싱 전환에 맞춰 조건 교체).
+     *
+     * <p>종전 조건은 {@code 기관명 = '알수없음' AND 분류상태구분 = '02'} 였는데, 새 파싱 경로는
+     * 기관을 못 찾으면 이름을 "기관미상" 으로 두고 분류상태를 미분류로 남긴다. 그래서
+     * <b>기관영업분류구분이 NULL</b> 인 것을 기준으로 바꿨다 - 그 값이 곧 "미분류" 다.
+     */
+
    @GetMapping
    public String execute(Model model) {
-         List<UploadedFile> classifiedFiles = fileMapper.findByStatus("CLASSIFIED");
-
-         long unclassified = fileMapper.countByStatus("UNCLASSIFIED");

```

```

+ List<UploadedFile> classifiedFiles =

+ fileMapper.searchParseStatus(null, "SUCCESS", Boolean.TRUE);

    model.addAttribute("classifiedFiles", classifiedFiles);
    model.addAttribute("categories", InstitutionCategory.names());
- model.addAttribute("unclassifiedCount", unclassified);

+ model.addAttribute("parseFailedCount", fileMapper.countByParseStatus("FAILED"));

    return "classified-rework";
}
}

```

controller/KGI12200\$FileStatusDelete.java

```

@@ -22,7 +22,7 @@ public class KGI12200$FileStatusDelete {

    @PostMapping("/{id}/delete")
    public String execute(@PathVariable Long id, RedirectAttributes ra) {
        fileMapper.softDeleteById(id, SystemUser.get(), LocalDateTime.now());
        ra.addFlashAttribute("message", "파일이 목록에서 제거되었습니다.");
- return "redirect:/file-status";

+ return "redirect:/parse-status";

    }
}

```

controller/KGI12300\$FileStatusReject.java

```

@@ -22,7 +22,7 @@ public class KGI12300$FileStatusReject {

    @PostMapping("/{id}/reject")
    public String execute(@PathVariable Long id, RedirectAttributes ra) {
        fileMapper.rejectById(id, SystemUser.get(), LocalDateTime.now());
        ra.addFlashAttribute("message", "반려 처리되었습니다.");
- return "redirect:/file-status";

+ return "redirect:/parse-status";

    }
}

```

controller/ProposalApiController.java

제안서 API (신규)

```

new file mode 100644
@@ -0,0 +1,120 @@

+package com.kb.uploader.controller;

+
+import com.fasterxml.jackson.databind.JsonNode;
+import com.fasterxml.jackson.databind.ObjectMapper;
+import com.fasterxml.jackson.databind.node.ObjectNode;
+import com.kb.uploader.code.DocumentType;
+import com.kb.uploader.domain.UploadedFile;

```

```

+import com.kb.uploader.mapper.UploadedFileMapper;

+import org.springframework.http.HttpStatus;

+import org.springframework.http.ResponseEntity;

+import org.springframework.util.StringUtils;

+import org.springframework.web.bind.annotation.*;

+

+import java.io.IOException;

+import java.nio.file.Files;

+import java.nio.file.Path;

+import java.nio.file.Paths;

+import java.time.format.DateTimeFormatter;

+import java.util.ArrayList;

+import java.util.Collections;

+import java.util.LinkedHashMap;

+import java.util.List;

+import java.util.Map;

+import java.util.Optional;

+

+/**
+ * 입찰제안서 조회 REST API (외부 agent 호출용, 인증 없음).
+ *
+ * <p>같은 저장소의 {@code FileSearchApiController} 와 같은 방식이라 화면번호를 쓰지 않는다.
+ */
+@RestController
+@RequestMapping("/api/proposals")
+public class ProposalApiController {

+

+    private static final DateTimeFormatter TS = DateTimeFormatter.ofPattern("yyyy-MM-dd HH:mm:ss");

+

+    private final UploadedFileMapper fileMapper;

+    private final ObjectMapper mapper;

+

+    public ProposalApiController(UploadedFileMapper fileMapper, ObjectMapper mapper) {

+        this.fileMapper = fileMapper;

+        this.mapper = mapper;

```

```

+     }
+
+     /** 파싱에 성공한 입찰제안서 목록. 모든 조건은 선택. */
+     @GetMapping
+     public Map<String, Object> search(@RequestParam(required = false) String institution,
+
+                                     @RequestParam(required = false) String year,
+
+                                     @RequestParam(defaultValue = "1") int page,
+
+                                     @RequestParam(defaultValue = "20") int size) {
+
+         int p = Math.max(page, 1);
+
+         int s = Math.min(Math.max(size, 1), 100);
+
+         // ⚠ Oracle 은 빈 문자열을 NULL 로 취급한다. 조건이 비면 null 을 넘긴다(기존 search 와 같은 규칙).
+
+         String inst = trimToNull(institution);
+
+         String docDate = trimToNull(year);
+
+
+
+         List<UploadedFile> rows = fileMapper.searchProposals(inst, docDate, (p - 1) * s, s);
+
+         long total = fileMapper.countProposals(inst, docDate);
+
+
+
+         List<Map<String, Object>> items = new ArrayList<Map<String, Object>>();
+
+         for (UploadedFile f : rows) items.add(summary(f));
+
+
+
+         Map<String, Object> body = new LinkedHashMap<String, Object>();
+
+         body.put("total", total);
+
+         body.put("page", p);
+
+         body.put("size", s);
+
+         body.put("totalPages", (int) Math.ceil((double) total / s));
+
+         body.put("items", items);
+
+         return body;
+     }
+
+
+     /** 제안서 한 건의 파싱 JSON 전체 (slides 포함). */
+     @GetMapping("/{id}")
+     public ResponseEntity<?> detail(@PathVariable Long id) throws IOException {
+
+         Optional<UploadedFile> found = fileMapper.findById(id);
+
+         if (!found.isPresent() || !DocumentType.BID_PROPOSAL.equals(found.get().getDocType())) {
+
+             return error(HttpStatus.NOT_FOUND, "입찰제안서를 찾을 수 없습니다 (id=" + id + ")");

```

```

+     }
+
+     UploadedFile f = found.get();
+
+     if (!"SUCCESS".equals(f.getParseStatus()) || f.getOutputPath() == null) {
+
+         return error(HttpStatus.NOT_FOUND, "파싱에 성공하지 않은 제안서입니다 (id=" + id + ", 상태="
+
+             + f.getParseStatus() + ")");
+
+     }
+
+     Path json = Paths.get(f.getOutputPath());
+
+     if (!Files.exists(json)) {
+
+         return error(HttpStatus.NOT_FOUND, "파싱 JSON 파일이 없습니다: " + json.getFileName());
+
+     }
+
+     JsonNode parsed = mapper.readTree(json.toFile());
+
+     ObjectNode body = mapper.createObjectNode();
+
+     body.put("id", f.getId());
+
+     body.put("institutionName", f.getInstitutionName());
+
+     body.put("institutionCategory", f.getCategoryLabel());
+
+     body.put("year", f.getDocDate());
+
+     body.put("jsonFileName", json.getFileName().toString());
+
+     body.setAll((ObjectNode) parsed);
+
+     return ResponseEntity.ok(body);
+
+ }
+
+
+ private static Map<String, Object> summary(UploadedFile f) {
+
+     Map<String, Object> m = new LinkedHashMap<String, Object>();
+
+     m.put("id", f.getId());
+
+     m.put("fileName", f.getOriginalName());
+
+     m.put("institutionName", f.getInstitutionName());
+
+     m.put("institutionCategory", f.getCategoryLabel());
+
+     m.put("year", f.getDocDate());
+
+     m.put("totalSlides", f.getPageCount());
+
+     m.put("jsonFileName", f.getOutputFileName());
+
+     m.put("uploadedAt", f.getUploadedAt() != null ? f.getUploadedAt().format(TS) : null);
+
+     m.put("parsedAt", f.getParsedAt() != null ? f.getParsedAt().format(TS) : null);
+
+     return m;
+
+ }
+

```

```

+ private static ResponseEntity<Map<String, String>> error(HttpStatus status, String message) {
+     return ResponseEntity.status(status).body(Collections.singletonMap("error", message));
+ }
+
+ private static String trimToNull(String s) {
+     return StringUtils.hasText(s) ? s.trim() : null;
+ }
+
+}

```

controller/RfpApiController.java

RFP API (신규) ?format=text 면 md 원문을 그대로 준다.

```

new file mode 100644
@@ -0,0 +1,141 @@
+package com.kb.uploader.controller;
+
+import com.kb.uploader.code.DocumentType;
+import com.kb.uploader.domain.UploadedFile;
+import com.kb.uploader.mapper.UploadedFileMapper;
+import org.springframework.http.HttpHeaders;
+import org.springframework.http.HttpStatus;
+import org.springframework.http.MediaType;
+import org.springframework.http.ResponseEntity;
+import org.springframework.util.StringUtils;
+import org.springframework.web.bind.annotation.*;
+
+import java.io.IOException;
+import java.nio.charset.StandardCharsets;
+import java.nio.file.Files;
+import java.nio.file.Path;
+import java.nio.file.Paths;
+import java.time.format.DateTimeFormatter;
+import java.util.ArrayList;
+import java.util.Collections;
+import java.util.LinkedHashMap;
+import java.util.List;

```

```

+import java.util.Map;

+import java.util.Optional;

+

+/**
+ * 입찰공고문(RFP) 요약 조회 REST API (외부 agent 호출용, 인증 없음).
+ *
+ * <p>제안서는 산출물이 JSON 이라 그대로 내려주면 되지만 RFP 산출물은 Markdown 이라 쓰는 쪽이 galinda.
+ * 그래서 같은 주소에서 두 가지로 준다.
+ * <ul>
+ *   <li>기본(JSON): 메타데이터 + {@code markdown} 필드에 원문 문자열</li>
+ *   <li>{@code ?format=text} 또는 {@code Accept: text/markdown}: md 원문 그대로 (LLM 입력용)</li>
+ * </ul>
+ */
+@RestController
+@RequestMapping("/api/rfps")
+public class RfpApiController {
+
+
+    private static final DateTimeFormatter TS = DateTimeFormatter.ofPattern("yyyy-MM-dd HH:mm:ss");
+
+    private static final MediaType MARKDOWN = new MediaType("text", "markdown", StandardCharsets.UTF_8);
+
+
+    private final UploadedFileMapper fileMapper;
+
+
+    public RfpApiController(UploadedFileMapper fileMapper) {
+
+        this.fileMapper = fileMapper;
+
+    }
+
+
+    /** 파싱에 성공한 RFP 목록. 모든 조건은 선택. */
+
+    @GetMapping
+
+    public Map<String, Object> search(@RequestParam(required = false) String institution,
+
+                                     @RequestParam(required = false) String noticeDate,
+
+                                     @RequestParam(defaultValue = "1") int page,
+
+                                     @RequestParam(defaultValue = "20") int size) {
+
+        int p = Math.max(page, 1);
+
+        int s = Math.min(Math.max(size, 1), 100);
+
+        String inst = trimOrNull(institution);

```

```

+     String docDate = trimToNull(noticeDate);
+
+
+     List<UploadedFile> rows = fileMapper.searchRfps(inst, docDate, (p - 1) * s, s);
+
+     long total = fileMapper.countRfps(inst, docDate);
+
+
+     List<Map<String, Object>> items = new ArrayList<Map<String, Object>>();
+
+     for (UploadedFile f : rows) items.add(summary(f));
+
+
+     Map<String, Object> body = new LinkedHashMap<String, Object>();
+
+     body.put("total", total);
+
+     body.put("page", p);
+
+     body.put("size", s);
+
+     body.put("totalPages", (int) Math.ceil((double) total / s));
+
+     body.put("items", items);
+
+     return body;
+ }
+
+
+ /**
+  * RFP 한 건의 요약.
+  *
+  * @param format {@code json}(기본) 이면 메타데이터 + markdown 필드, {@code text} 면 md 원문
+  */
+ @GetMapping("/{id}")
+ public ResponseEntity<?> detail(@PathVariable Long id,
+
+     @RequestParam(required = false) String format,
+
+     @RequestHeader(name = HttpHeaders.ACCEPT, required = false) String accept)
+
+     throws IOException {
+
+     Optional<UploadedFile> found = fileMapper.findById(id);
+
+     if (!found.isPresent() || !DocumentType.RFP.equals(found.get().getDocType())) {
+
+         return error(HttpStatus.NOT_FOUND, "입찰공고문(RFP)을 찾을 수 없습니다 (id=" + id + ")");
+
+     }
+
+     UploadedFile f = found.get();
+
+     if (!"SUCCESS".equals(f.getParseStatus()) || f.getOutputPath() == null) {
+
+         return error(HttpStatus.NOT_FOUND, "파싱에 성공하지 않은 공고문입니다 (id=" + id + ", 상태="
+
+             + f.getParseStatus() + ")");
+
+     }

```

```

+     }
+
+     Path md = Paths.get(f.getOutputPath());
+
+     if (!Files.exists(md)) {
+
+         return error(HttpStatus.NOT_FOUND, "요약 MD 파일이 없습니다: " + md.getFileName());
+
+     }
+
+     String markdown = new String(Files.readAllBytes(md), StandardCharsets.UTF_8);
+
+
+     if (wantsRawMarkdown(format, accept)) {
+
+         return ResponseEntity.ok().contentType(MARKDOWN).body(markdown);
+
+     }
+
+     Map<String, Object> body = summary(f);
+
+     body.put("markdown", markdown);
+
+     return ResponseEntity.ok(body);
+
+ }
+
+
+ /** {@code ?format=text} 가 우선이고, 없으면 Accept 헤더가 md/plain 만 받겠다고 한 경우. */
+ private static boolean wantsRawMarkdown(String format, String accept) {
+
+     if (StringUtils.hasText(format)) {
+
+         String f = format.trim().toLowerCase();
+
+         return "text".equals(f) || "md".equals(f) || "markdown".equals(f);
+
+     }
+
+     if (!StringUtils.hasText(accept)) return false;
+
+     String a = accept.toLowerCase();
+
+     if (a.contains("application/json") || a.contains("*/")) return false;
+
+     return a.contains("text/markdown") || a.contains("text/plain");
+
+ }
+
+
+ private static Map<String, Object> summary(UploadedFile f) {
+
+     Map<String, Object> m = new LinkedHashMap<String, Object>();
+
+     m.put("id", f.getId());
+
+     m.put("fileName", f.getOriginalName());
+
+     m.put("institutionName", f.getInstitutionName());
+
+     m.put("institutionCategory", f.getCategoryLabel());
+
+     m.put("noticeDate", f.getDocDate());
+
+     m.put("pageCount", f.getPageCount());

```

```
+     m.put("mdFileName", f.getOutputFileName());

+     m.put("uploadedAt", f.getUploadedAt() != null ? f.getUploadedAt().format(TS) : null);

+     m.put("parsedAt", f.getParsedAt() != null ? f.getParsedAt().format(TS) : null);

+     return m;

+ }

+

+ private static ResponseEntity<Map<String, String>> error(HttpStatus status, String message) {

+     return ResponseEntity.status(status).body(Collections.singletonMap("error", message));

+ }

+

+ private static String trimToNull(String s) {

+     return StringUtils.hasText(s) ? s.trim() : null;

+ }

+ }
```

9. 화면

templates/parse-status.html

파싱 현황 화면 (신규) file-status.html 을 대신한다.

```
new file mode 100644
@@ -0,0 +1,118 @@

+<!DOCTYPE html>

+<html xmlns:th="http://www.thymeleaf.org"
+  th:replace="layout :: layout(~{::title}, '파싱 현황', ~{::section})">
+<head><title>파싱 현황</title></head>
+<body>
+<section>
+
+  <div class="section-header" style="margin-bottom:20px">
+    <div style="display:flex;align-items:center;gap:10px;flex-wrap:wrap">
+      <span class="section-title" style="font-size:16px">파싱 현황</span>
+      <span class="badge badge-done" th:text="'성공 ' + ${successCount}"></span>
+      <span th:if="${parseFailedCount > 0}" class="badge badge-pending"
+        th:text="'실패 ' + ${parseFailedCount}"></span>
+      <span th:if="${unclassifiedCount > 0}" class="badge badge-neutral"
+        th:text="'기관 미분류 ' + ${unclassifiedCount}"></span>
+    </div>
+  </div>
+
+  <div th:if="${message}" class="alert-success" th:text="${message}"></div>
+  <div th:if="${error}" class="alert-error" th:text="${error}"></div>
+
+  <!-- 필터 -->
+  <form method="get" th:action="@{/parse-status}" class="search-bar">
+    <select name="docType" class="form-control" style="max-width:180px">
+      <option value="">문서종류 전체</option>
+      <option th:each="t : ${docTypes}" th:value="${t}"
+        th:text="${t == 'BID_PROPOSAL' ? '입찰제안서' : '입찰공고문(RFP)'}"
+        th:selected="${docType == t}"></option>
```

```

+   </select>
+   <select name="parseStatus" class="form-control" style="max-width:160px">
+     <option value="">파싱상태 전체</option>
+     <option value="SUCCESS" th:selected="${parseStatus == 'SUCCESS'}">성공</option>
+     <option value="FAILED" th:selected="${parseStatus == 'FAILED'}">실패</option>
+     <option value="PENDING" th:selected="${parseStatus == 'PENDING'}">대기</option>
+   </select>
+   <select name="category" class="form-control" style="max-width:160px">
+     <option value="">기관분류 전체</option>
+     <option value="미분류" th:selected="${category == '미분류'}">미분류만</option>
+   </select>
+   <button type="submit" class="btn btn-primary">조회</button>
+   <a th:href="@{/parse-status}" class="btn btn-outline">초기화</a>
+ </form>

```

```

+
+ <div th:if="${files.isEmpty()}" class="empty-state">
+   <svg class="icon" style="width:40px;height:40px;opacity:.35" viewBox="0 0 24 24">
+     <path d="M14 2H6a2 2 0 0 0-2 2v16a2 2 0 0 0 2 2h12a2 2 0 0 0 2-2V8z"/>
+     <polyline points="14 2 14 8 20 8"/>
+   </svg>
+   <div class="empty-state-text">조건에 맞는 파일이 없습니다.</div>
+ </div>

```

```

+ <div th:if="${!files.isEmpty()}" class="table-card">
+   <table>
+     <thead>
+       <tr>
+         <th>문서종류</th>
+         <th>원본 파일명</th>
+         <th>기관</th>
+         <th>기관분류</th>
+         <th>연도 / 공고일</th>
+         <th>파싱</th>
+         <th>산출물</th>
+         <th>파싱 일시</th>

```

```
+      <th></th>
+    </tr>
+  </thead>
+  <tbody>
+    <tr th:each="f : ${files}">
+      <td style="font-size:12.5px" th:text="${f.docTypeLabel}"></td>
+      <td>
+        <span th:with="ext=${#strings.toLowerCase(f.originalName.substring(f.originalName.lastIndexOf('.') + 1))}"
+          th:class="'file-ext ext-' + ${ext}" th:text="${#strings.toUpperCase(ext)}"></span>
+        <span class="file-name" th:text="${f.originalName}"></span>
+      </td>
+      <td style="font-size:12.5px" th:text="${f.institutionName != null ? f.institutionName : '-'}"></td>
+      <td>
+        <span class="category-chip" th:text="${f.categoryLabel}"></span>
+      </td>
+      <td class="time-col" th:text="${f.docDate != null ? f.docDate : '-'}"></td>
+      <td>
+        <span th:if="${f.parseStatus == 'SUCCESS'}" class="badge badge-done">성공</span>
+        <span th:if="${f.parseStatus == 'FAILED'}" class="badge badge-pending">실패</span>
+        <span th:if="${f.parseStatus != 'SUCCESS' and f.parseStatus != 'FAILED'}"
+          class="badge badge-neutral" th:text="${f.parseStatus != null ? f.parseStatus : '-'}"></span>
+        <div th:if="${f.parseMessage != null}" style="font-size:11.5px;color:var(--text-2);margin-top:3px"
+          th:text="${f.parseMessage}"></div>
+      </td>
+      <td>
+        <a th:if="${f.outputPath != null}" th:href="@{/parse-status/{id}/output(id=${f.id})}"
+          target="_blank" class="file-name" style="font-size:12.5px"
+          th:text="${f.outputFileName}"></a>
+        <span th:if="${f.outputPath == null}" style="color:var(--text-3)">-</span>
+      </td>
+      <td class="time-col"
+        th:text="${f.parsedAt != null ? #temporals.format(f.parsedAt, 'yyyy-MM-dd HH:mm') : '-'}"></td>
+      <td>
+        <div class="form-inline">
+          <a th:if="${f.outputPath != null}"
```

```

+         th:href="@{/parse-status/{id}/output(id=${f.id},download=true)}"
+         class="btn btn-outline btn-sm">내려받기</a>
+         <form th:action="@{/parse-status/{id}/reparse(id=${f.id})}" method="post">
+             <button type="submit" class="btn btn-primary btn-sm"
+                 th:disabled="${f.docType == null}">재파싱</button>
+         </form>
+     </div>
+ </td>
+ </tr>
+ </tbody>
+ </table>
+ </div>
+
+ <p style="font-size:12.5px;color:var(--text-2);margin-top:12px">
+     기관을 새로 등록한 뒤 <b>재파싱</b>을 누르면 기관분류가 반영되고 산출물 파일명도 새로 만들어집니다.
+ </p>
+
+ </section>
+</body>
+</html>

```

templates/layout.html

공통 레이아웃 좌측 네비 2곳과 agent 테스트 패널(CSS-마크업-스크립트 태그).

```

@@ -384,9 +384,98 @@
    /* — Inline form — */
    .form-inline { display: flex; gap: 6px; align-items: center; flex-wrap: wrap; }
    /* — SVG icon — */
    .icon { width:16px; height:16px; stroke:currentColor; fill:none; stroke-width:1.8; stroke-linecap:round; stroke-linejoin:round; }
- </style>
+
+ /* — agent 테스트 패널 — */
+
+ .agent-panel {
+     position: fixed; top: 0; right: 0; bottom: 0; width: 480px; max-width: 100vw;
+     background: var(--surface); border-left: 1px solid var(--border);
+     box-shadow: var(--shadow-high); transform: translateX(100%);
+     transition: transform .18s ease; z-index: 120; display: flex; flex-direction: column;
+ }
+
+ .agent-panel.open { transform: translateX(0); }

```

```
+ .agent-panel-head {  
  
+   display: flex; align-items: center; gap: 8px; padding: 14px 16px;  
  
+   border-bottom: 1px solid var(--border); background: var(--kb-navy); color: #fff;  
  
+ }  
  
+ .agent-panel-head .title { font-size: 14px; font-weight: 600; flex: 1; }  
  
+ .agent-close {  
  
+   background: transparent; border: 0; color: inherit; cursor: pointer;  
  
+   font-size: 20px; line-height: 1; padding: 2px 6px; border-radius: var(--r-sm);  
  
+ }  
  
+ .agent-close:hover { background: rgba(255,255,255,.15); }  
  
+ .agent-panel-body { padding: 16px; overflow-y: auto; flex: 1; }  
  
+ .agent-section-title { font-size: 12px; font-weight: 700; color: var(--text-3); letter-spacing: .04em; margin: 0 0 8px; text-transform: uppercase; }  
  
+ .agent-api-item {  
  
+   border: 1px solid var(--border); border-radius: var(--r-md); padding: 10px 12px;  
  
+   cursor: pointer; margin-bottom: 8px; background: var(--bg);  
  
+ }  
  
+ .agent-api-item.selected { border-color: var(--kb-navy); background: var(--surface); box-shadow: 0 0 0 2px rgba(27,42,107,.1); }  
  
+ .agent-api-item .name { font-size: 13px; font-weight: 600; color: var(--text-1); }  
  
+ .agent-api-item .path { font-family: var(--mono); font-size: 11.5px; color: var(--text-2); margin-top: 2px; word-break: break-all; }  
  
+ .agent-field { display: flex; flex-direction: column; gap: 4px; margin-bottom: 10px; }  
  
+ .agent-field label { font-size: 12px; color: var(--text-2); font-weight: 500; }  
  
+ .agent-url {  
  
+   font-family: var(--mono); font-size: 11.5px; color: var(--text-2);  
  
+   background: var(--bg); border: 1px dashed var(--border); border-radius: var(--r-sm);  
  
+   padding: 6px 8px; word-break: break-all; margin: 8px 0 12px;  
  
+ }  
  
+ .agent-status { display: flex; gap: 8px; align-items: center; margin-bottom: 10px; font-size: 12px; }  
  
+ .agent-pill { display: inline-flex; padding: 2px 8px; border-radius: 999px; font-weight: 700; font-size: 11.5px; }  
  
+ .agent-pill.ok { background: var(--green-bg); color: var(--green); }  
  
+ .agent-pill.err { background: var(--red-bg); color: var(--red); }  
  
+ .agent-json {  
  
+   font-family: var(--mono); font-size: 11.5px; white-space: pre; overflow: auto;  
  
+   max-height: 320px; background: var(--bg); border: 1px solid var(--border);
```

```

+ border-radius: var(--r-sm); padding: 10px; color: var(--text-1);
+}

+/* — 대화형 요청 로그 — */

+.agent-chat { display: flex; flex-direction: column; gap: 8px; margin-bottom: 14px; }

+.agent-bubble {
+ max-width: 88%; padding: 8px 11px; border-radius: 12px; font-size: 12.5px; line-height: 1.55;
+ white-space: pre-wrap; word-break: break-word;
+}

+.agent-bubble.me { align-self: flex-end; background: var(--kb-navy); color: #fff; border-bottom-right-radius: 4px; }

+.agent-bubble.bot { align-self: flex-start; background: var(--bg); border: 1px solid var(--border); color: var(--text-1);
border-bottom-left-radius: 4px; }

+.agent-caret { display: inline-block; width: 6px; color: inherit; animation: agentCaret 1s steps(1) infinite; }

+@keyframes agentCaret { 50% { opacity: 0; } }

+.agent-dots { display: inline-flex; gap: 4px; align-items: center; padding: 2px 0; }

+.agent-dots i {
+ width: 5px; height: 5px; border-radius: 50%; background: var(--text-3); display: block;
+ animation: agentDot 1.1s ease-in-out infinite;
+}

+.agent-dots i:nth-child(2) { animation-delay: .18s; }

+.agent-dots i:nth-child(3) { animation-delay: .36s; }

+@keyframes agentDot { 0%, 60%, 100% { opacity: .25; transform: translateY(0); } 30% { opacity: 1; transform: translateY(-3px); } }

+@media (prefers-reduced-motion: reduce) {
+ .agent-caret, .agent-dots i { animation: none; }
+}

+.agent-result-table { width: 100%; border-collapse: collapse; font-size: 12.5px; margin-bottom: 12px; }

+.agent-result-table th, .agent-result-table td { border-bottom: 1px solid var(--border); padding: 6px 4px; text-align: left; vertical-align: top; }

+.agent-result-table th { color: var(--text-3); font-weight: 600; font-size: 11.5px; }

+/* — 크게보기 모달 — */

+.agent-overlay { position: fixed; inset: 0; background: rgba(15,28,77,.45); z-index: 130; display: none; }

+.agent-overlay.open { display: block; }

+.agent-modal {
+ position: fixed; inset: 24px; z-index: 140; background: var(--surface);
+ border-radius: var(--r-lg); box-shadow: var(--shadow-high); display: none;
+ flex-direction: column; overflow: hidden;
+}

```

```

+.agent-modal.open { display: flex; }

+.agent-modal-head { display: flex; align-items: center; gap: 12px; padding: 14px 18px; border-bottom: 1px solid var(--border); }

+.agent-modal-head .title { font-size: 15px; font-weight: 700; color: var(--text-1); flex: 1; }

+.agent-modal-meta { font-size: 12px; color: var(--text-2); display: flex; gap: 10px; flex-wrap: wrap; padding: 10px 18px; border-bottom: 1px solid var(--border); background: var(--bg); }

+.agent-tabs { display: flex; gap: 4px; padding: 10px 18px 0; }

+.agent-tab { border: 1px solid var(--border); border-bottom: 0; background: var(--bg); color: var(--text-2); padding: 6px 14px; border-radius: var(--r-sm) var(--r-sm) 0 0; cursor: pointer; font-size: 12.5px; }

+.agent-tab.active { background: var(--surface); color: var(--text-1); font-weight: 600; }

+.agent-modal-body { padding: 16px 18px; overflow-y: auto; flex: 1; border-top: 1px solid var(--border); }

+.slide-card { border: 1px solid var(--border); border-radius: var(--r-md); padding: 12px 14px; margin-bottom: 10px; background: var(--surface); }

+.slide-card-head { display: flex; align-items: center; gap: 8px; margin-bottom: 6px; }

+.slide-no { font-family: var(--mono); font-size: 11.5px; color: #fff; background: var(--kb-navy); border-radius: var(--r-sm); padding: 2px 7px; }

+.slide-title { font-size: 13.5px; font-weight: 600; color: var(--text-1); flex: 1; }

+.slide-text { font-size: 12.5px; color: var(--text-2); white-space: pre-wrap; line-height: 1.6; }

+</style>

</head>
<body>

<!-- — 사이드바 — -->
@@ -423,26 +512,27 @@

    </svg>
    파일 업로드
  </a>

-   <a th:href="@{/file-status}"
-       th:class="{ 'nav-item' + (#request.getRequestURI() == '/file-status' ? ' active' : '') }">
+   <a th:href="@{/parse-status}"
+       th:class="{ 'nav-item' + (#request.getRequestURI() == '/parse-status' ? ' active' : '') }">

      <svg class="nav-icon icon" viewBox="0 0 24 24">
        <path d="M14 2H6a2 2 0 0 0-2 2v16a2 2 0 0 0 2 2h12a2 2 0 0 0 2-2V8z"/>
        <polyline points="14 2 14 8 20 8"/>
      </svg>
-     파일 처리 상태

-     <span th:if="{unclassifiedCount != null and unclassifiedCount > 0}"
-         class="nav-badge" th:text="{unclassifiedCount}"></span>
+     파싱 현황

+     <!-- 2026-09-23: 배지가 미분류 건수 → 파싱 실패 건수로 바뀌었다 -->

+     <span th:if="{parseFailedCount != null and parseFailedCount > 0}"

```

```
+         class="nav-badge" th:text="${parseFailedCount}"></span>

</a>

<a th:href="@{/file-status/classified}"
  th:class="${'nav-item' + (#request.getRequestURI().startsWith('/file-status/classified') ? ' active' : '')}">
  <svg class="nav-icon icon" viewBox="0 0 24 24">
    <path d="M11 4H4a2 2 0 0 0-2 2v14a2 2 0 0 0 2 2h14a2 2 0 0 0 2-2v-7"/>
    <path d="M18.5 2.5a2.121 2.121 0 0 1 3 3l12 15l-4 1 1-4 9.5-9.5z"/>
  </svg>
-   분류 재작업

+   기관 미분류

</a>

<a th:href="@{/institutions}"
  th:class="${'nav-item' + (#request.getRequestURI().startsWith('/institutions') ? ' active' : '')}">
@@ -461,8 +551,15 @@

<main class="main">
  <header class="topbar">
    <span class="topbar-title" th:text="${pageTitle}">파일 업로드 관리</span>
    <div class="topbar-divider"></div>
+   <button type="button" class="btn btn-outline btn-sm" id="agentTestOpen">

+   <svg class="icon" viewBox="0 0 24 24" style="margin-right:4px">

+   <polyline points="16 18 22 12 16 6"/>

+   <polyline points="8 6 2 12 8 18"/>

+   </svg>

+   agent 테스트

+   </button>

    <div class="topbar-user">
      <div class="avatar">관</div>
      관리자
    </div>
@@ -472,6 +569,40 @@

  </div>
</main>

  <script th:src="@{/js/bootstrap.bundle.min.js}"></script>
+<!-- — agent 테스트 패널 — -->

+<aside class="agent-panel" id="agentPanel" th:attr="data-base=@{/}">

+  <div class="agent-panel-head">

+    <span class="title">agent 테스트 — 제공 API 호출</span>

+    <button type="button" class="agent-close" id="agentTestClose" aria-label="닫기">&times;</button>

+  </div>

+  <div class="agent-panel-body">

+    <p class="agent-section-title">API 목록</p>

+    <div id="agentApiList"></div>

+

+    <p class="agent-section-title" style="margin-top:18px">파라미터</p>
```

```

+   <div id="agentParams"></div>
+   <div class="agent-url" id="agentUrl"></div>
+   <button type="button" class="btn btn-primary btn-sm" id="agentSend">요청</button>
+
+   <div id="agentResult" style="margin-top:16px"></div>
+ </div>
+</aside>
+
+<div class="agent-overlay" id="agentOverlay"></div>
+<div class="agent-modal" id="agentModal">
+  <div class="agent-modal-head">
+    <span class="title" id="agentModalTitle">제안서</span>
+    <button type="button" class="btn btn-outline btn-sm" id="agentModalClose">닫기</button>
+  </div>
+  <div class="agent-modal-meta" id="agentModalMeta"></div>
+  <!-- 탭은 문서 종류(제안서/RFP)에 따라 agent-test.js 가 채운다 -->
+  <div class="agent-tabs" id="agentModalTabs">
+  </div>
+  <div class="agent-modal-body" id="agentModalBody"></div>
+</div>
+
+<script th:src="@{/js/bootstrap.bundle.min.js}"></script>
+<script th:src="@{/js/agent-test.js}"></script>
+
+</body>
+</html>

```

templates/dashboard.html

대시보드 화면 KPI 4개와 경고 배너.

```

@@ -15,9 +15,9 @@
    </div>
    <div class="kpi-body">
      <div class="kpi-label">전체 파일</div>
      <div class="kpi-value" th:text="${total}">0</div>
-     <div class="kpi-sub">누적 업로드</div>
+
+     <div class="kpi-sub" th:text="'제안서 ' + ${proposalCount} + ' · RFP ' + ${rfpCount}'">누적 업로드</div>
+
    </div>
  </div>

  <div class="kpi-card">

```

@@ -26,11 +26,11 @@

```
        <polyline points="20 6 9 17 4 12"/>
    </svg>
</div>
<div class="kpi-body">
-     <div class="kpi-label">분류 완료</div>
-
-     <div class="kpi-value success" th:text="${classified}">0</div>
-
-     <div class="kpi-sub">기관 분류됨</div>
+
+     <div class="kpi-label">파싱 성공</div>
+
+     <div class="kpi-value success" th:text="${parseSuccess}">0</div>
+
+     <div class="kpi-sub">산출물 생성됨</div>
</div>
</div>
```

```
<div class="kpi-card">
```

@@ -41,11 +41,11 @@

```
        <line x1="12" y1="16" x2="12.01" y2="16"/>
    </svg>
</div>
<div class="kpi-body">
-     <div class="kpi-label">미분류</div>
-
-     <div th:class="${'kpi-value' + (unclassified > 0 ? ' danger' : '')}" th:text="${unclassified}">0</div>
-
-     <div class="kpi-sub">즉시 처리 필요</div>
+
+     <div class="kpi-label">파싱 실패</div>
+
+     <div th:class="${'kpi-value' + (parseFailed > 0 ? ' danger' : '')}" th:text="${parseFailed}">0</div>
+
+     <div class="kpi-sub">재파싱 필요</div>
</div>
</div>
```

```
<div class="kpi-card">
```

@@ -55,24 +55,24 @@

```
        <line x1="4.93" y1="4.93" x2="19.07" y2="19.07"/>
    </svg>
</div>
<div class="kpi-body">
-     <div class="kpi-label">반려 / 삭제</div>
-
-     <div class="kpi-value" style="color:var(--text-3,#999)" th:text="${excluded}">0</div>
-
-     <div class="kpi-sub">처리 제외됨</div>
+
+     <div class="kpi-label">기관 미분류</div>
+
+     <div class="kpi-value" style="color:var(--text-3,#999)" th:text="${unclassified}">0</div>
+
+     <div class="kpi-sub">기관 등록 후 재파싱</div>
</div>
</div>
</div>
```

```
<!-- 경고 배너 (미분류 > 0 일 때만) -->
```

```
- <div th:if="${unclassified > 0}" class="alert-banner">
```

```

+ <div th:if="{parseFailed > 0}" class="alert-banner">

    <svg class="icon" style="width:15px;height:15px;flex-shrink:0" viewBox="0 0 24 24">
      <path d="M10.29 3.86L1.82 18a2 2 0 0 0 1.71 3h16.94a2 2 0 0 0 1.71-3L13.71 3.86a2 2 0 0 0-3.42 0z"/>
      <line x1="12" y1="9" x2="12" y2="13"/>
      <line x1="12" y1="17" x2="12.01" y2="17"/>
    </svg>
-   미분류 파일 <strong th:text="{unclassified}"></strong>건이 처리 대기 중입니다.

-   <a th:href="@{/file-status}" style="font-weight:700;margin-left:4px;color:inherit;text-decoration:underline">파일 처리
상태 바로가기</a>

+   파싱 실패 <strong th:text="{parseFailed}"></strong>건이 있습니다.

+   <a th:href="@{/parse-status(parseStatus='FAILED')}" style="font-weight:700;margin-left:4px;color:inherit;text-decoration:underline">파싱
현황 바로가기</a>

</div>

<!-- 검색 -->
<div class="section-header">
@@ -108,10 +108,9 @@

        <span class="file-name" th:text="{f.originalName}"></span>
      </td>
      <td th:text="{f.institutionName}"></td>
      <td>
-        <span th:if="{f.category != null}" class="category-chip" th:text="{f.category}"></span>
-        <span th:if="{f.category == null}" style="color:var(--text-3)">-</span>
+        <span class="category-chip" th:text="{f.categoryLabel}"></span>

      </td>
      <td>
        <span th:if="{f.status == 'CLASSIFIED'}" class="badge badge-done">분류 완료</span>
        <span th:if="{f.status == 'UNCLASSIFIED'}" class="badge badge-pending">미분류</span>
@@ -161,10 +160,9 @@

        <span class="file-name" th:text="{f.originalName}"></span>
      </td>
      <td th:text="{f.institutionName}"></td>
      <td>
-        <span th:if="{f.category != null}" class="category-chip" th:text="{f.category}"></span>
-        <span th:if="{f.category == null}" style="color:var(--text-3)">-</span>
+        <span class="category-chip" th:text="{f.categoryLabel}"></span>

      </td>
      <td>
        <span th:if="{f.status == 'CLASSIFIED'}" class="badge badge-done">분류 완료</span>
        <span th:if="{f.status == 'UNCLASSIFIED'}" class="badge badge-pending">미분류</span>

```

templates/upload.html

업로드 화면 제안서/RFP 2분할.

```

@@ -6,77 +6,82 @@

<section>

  <div class="section-header" style="margin-bottom:20px">
    <span class="section-title" style="font-size:18px">파일 업로드</span>
+   <span class="section-sub">업로드하면 문서 내용을 읽어 산출물을 만듭니다. 기관·연도는 본문에서 찾습니다.</span>

  </div>

```

```

- <!-- 업로드 카드 -->

- <div class="card" style="max-width:600px;margin-bottom:24px">

- <div class="card-header">파일 선택</div>

- <div class="card-body">

- <form method="post" th:action="@{/upload}" enctype="multipart/form-data">

- <div class="form-group">

- <label class="form-label">파일 선택 (pdf, hwp, md - 복수 선택 가능)</label>

- <input type="file" name="files" multiple accept=".pdf,.hwp,.md" class="form-control">

- </div>

+ <div th:if="${errorMessage}"

+ style="margin-bottom:14px;padding:10px 14px;background:#fff3f3;border:1px solid #f5c6c6;border-radius:6px;color:#c0392b;font-size:13.5px"

+ th:text="${errorMessage}"></div>

- <!-- 수동 입력 (파일명 규칙 불일치 시) -->

- <div style="margin-top:16px;padding:12px 14px;background:var(--surface-2,#f8f9fa);border-radius:6px;border:1px solid var(--border,#e0e0e0)">

- <div style="color:var(--text-2,#666);margin-bottom:10px;font-weight:600">

- 파일명 규칙 불일치 시 직접 입력 <span style="font-weight:400">(선택사항 - 파일명이 <code>년도_기관명_설명.확장자</code> 형식이 아닐 때 사용)</span>

+ <div style="display:flex;gap:16px;flex-wrap:wrap;margin-bottom:24px">

+

+ <!-- 입찰제안서 -->

+ <div class="card" style="flex:1;min-width:320px">

+ <div class="card-header">입찰제안서</div>

+ <div class="card-body">

+ <form method="post" th:action="@{/upload/proposal}" enctype="multipart/form-data">

+ <div class="form-group">

+ <label class="form-label">ppt, pptx - 복수 선택 가능</label>

+ <input type="file" name="files" multiple accept=".ppt,.pptx" class="form-control">

+ </div>

- <div style="display:flex;gap:10px;flex-wrap:wrap">

- <div style="flex:1;min-width:140px">

- <label class="form-label">기관명</label>

- <input type="text" name="institution" class="form-control form-control-sm" placeholder="예: 서울에너지공사">

- </div>

- <div style="width:100px">

- <label class="form-label">연도</label>

```

```

-         <input type="text" name="year" class="form-control form-control-sm" placeholder="예: 2024" maxlength="4">
-     </div>
-     <div style="flex:1;min-width:140px">
-         <label class="form-label">카테고리</label>
-         <select name="category" class="form-control form-control-sm">
-             <option value="">선택 (선택사항)</option>
-             <option th:each="cat : ${categories}" th:value="${cat}" th:text="${cat}"></option>
-         </select>
-     </div>
+     <div style="color:var(--text-2,#666);font-size:12.5px;margin-top:8px">
+         슬라이드를 읽어 <code>{연도}_{기관분류}_{기관명}_{파싱일}_bid_proposal.json</code> 을 만듭니다.
+     </div>
- </div>
+     <button type="submit" class="btn btn-primary" style="margin-top:14px">제안서 업로드</button>
+ </form>
+ </div>
+ </div>

-     <div th:if="${errorMessage}" style="margin-top:12px;padding:10px 14px;background:#fff3f3;border:1px solid #f5c6c6;border-radius:6px;color:#c0392b;font-size:13.5px" th:text="${errorMessage}"></div>
-     <button type="submit" class="btn btn-primary" style="margin-top:14px">업로드</button>
- </form>

+ <!-- 입찰공고문 RFP -->
+ <div class="card" style="flex:1;min-width:320px">
+     <div class="card-header">입찰공고문 (RFP)</div>
+     <div class="card-body">
+         <form method="post" th:action="@{/upload/rfp}" enctype="multipart/form-data">
+             <div class="form-group">
+                 <label class="form-label">pdf, hwp, hwpX - 복수 선택 가능</label>
+                 <input type="file" name="files" multiple accept=".pdf,.hwp,.hwpX" class="form-control">
+             </div>
+             <div style="color:var(--text-2,#666);font-size:12.5px;margin-top:8px">
+                 본문을 요약해 <code>{공고일}_{기관분류}_{기관명}_{파싱일}.md</code> 를 만듭니다.
+             </div>
+             <button type="submit" class="btn btn-primary" style="margin-top:14px">공고문 업로드</button>
+         </form>

```

```

+     </div>

    </div>
+
</div>

<!-- 업로드 결과 -->
<div th:if="${results != null and !results.isEmpty()}">
    <div class="section-header">
        <span class="section-title">업로드 결과</span>
+        <span th:if="${uploadedType}" class="section-sub" th:text="${uploadedType}"></span>

    </div>
    <div class="table-card">
        <table>
            <thead>
                <tr>
-                    <th>파일명</th><th>카테고리</th><th>결과</th><th>메시지</th>
+                    <th>파일명</th><th>기관분류</th><th>파싱</th><th>메시지</th>

                </tr>
            </thead>
            <tbody>
                <tr th:each="r : ${results}">
                    <td>
-                        <span th:with="ext=${#strings.toLowerCase(r.filename.substring(r.filename.lastIndexOf('.') + 1))}"
-                            th:class="${'file-ext' + (ext == 'pdf' ? ' ext-pdf' : (ext == 'hwp' ? ' ext-hwp' : (ext == 'md' ? ' e
xt-md' : '')))}"
-                            th:text="${#strings.toUpperCase(ext)}"></span>
+                        <!-- ⚠ #strings.substringAfterLast 는 이 Thymeleaf 버전에 없다. 쓰면 렌더링이 끊긴다 -->
+                        <span th:with="ext=${#strings.toLowerCase(r.filename.substring(r.filename.lastIndexOf('.') + 1))}"
+                            th:class="${'file-ext' + (ext == 'pdf' ? ' ext-pdf' : (ext == 'hwp' ? ' ext-hwp' : (ext == 'md' ? ' ext
-md' : '')))}"
+                            th:text="${#strings.toUpperCase(ext)}"></span>

                        <span class="file-name" th:text="${r.filename}"></span>
                    </td>
                    <td>
                        <span th:if="${r.category != null}" class="category-chip" th:text="${r.category}"></span>
                        <span th:if="${r.category == null}" style="color:var(--text-3)"></span>
                    </td>
                    <td>
-                        <span th:class="${'badge ' + (r.classified ? 'badge-done' : 'badge-pending')}"
-                            th:text="${r.classified ? '분류 완료' : '미분류'}"></span>
+                        <span th:class="${'badge ' + (r.classified ? 'badge-done' : 'badge-pending')}"
+                            th:text="${r.classified ? '성공' : '실패'}"></span>

                    </td>
                    <td style="color:var(--text-2);font-size:12.5px" th:text="${r.message}"></td>
                </tr>
            </tbody>

```

templates/classified-rework.html

기관 미분류 화면 연도 → 문서일자, category → categoryLabel, 버튼은 "등록 후 재파싱".

@@ -46,16 +46,15 @@

```

<thead>
<tr>
  <th style="width:28%">파일명</th>
  <th style="width:16%">기관명</th>
-   <th style="width:7%">연도</th>
-
-   <th style="width:10%">카테고리</th>
-
-   <th style="width:39%">재분류</th>
+   <th style="width:9%">연도/공고일</th>
+   <th style="width:10%">기관분류</th>
+   <th style="width:37%">기관 지정 후 재파싱</th>

</tr>
</thead>
<tbody>
-   <tr th:each="file : ${classifiedFiles}"
-       th:style="${(file.institutionName == '알수없음' or file.year == null or file.year == '미확인') ? 'background:rgb
a(220,38,38,.03)' : ''}"
+   <tr th:each="file : ${classifiedFiles}" style="background:rgba(220,38,38,.03)">

    <!-- 파일명 -->
    <td>
      <span th:with="ext=${#strings.toLowerCase(file.originalName.substring(file.originalName.lastIndexOf('.') +
1)))}"
        th:class="${'file-ext' + (ext == 'pdf' ? ' ext-pdf' : (ext == 'hwp' ? ' ext-hwp' : (ext == 'md' ? ' ext
-md' : '')))}"
@@ -63,23 +62,19 @@
      <span class="file-name" th:text="${file.originalName}"></span>
    </td>
    <!-- 기관명 -->
    <td>
-       <span th:if="${file.institutionName == '알수없음'}"
-
-         style="color:var(--red);font-weight:600" th:text="${file.institutionName}"></span>
-
-       <span th:if="${file.institutionName != '알수없음'}"
-
-         style="color:var(--text-2)" th:text="${file.institutionName}"></span>
+       <span style="color:var(--red);font-weight:600"
+
+         th:text="${file.institutionName != null ? file.institutionName : '기관미상'}"></span>

    </td>
    <!-- 연도 -->
    <td>
-       <span th:if="${file.year == null or file.year == '미확인'}"
-
-         style="color:var(--amber);font-weight:600">미확인</span>
-
-       <span th:if="${file.year != null and file.year != '미확인'}"
-
-         style="color:var(--text-2)" th:text="${file.year}"></span>
+       <span th:if="${file.docDate == null}" style="color:var(--amber);font-weight:600">미확인</span>
+       <span th:if="${file.docDate != null}" style="color:var(--text-2)" th:text="${file.docDate}"></span>

    </td>
    <!-- 현재 카테고리 -->
    <td>

```

```

-      <span class="category-chip" th:text="${file.category}"></span>

+      <span class="category-chip" th:text="${file.categoryLabel}"></span>

    </td>
    <!-- 재분류 품 -->
    <td>
      <form class="rework-form"
@@ -88,17 +83,14 @@
        <select name="category" class="form-control form-control-sm" style="width:110px" required>
          <option th:each="cat : ${categories}"
            th:value="${cat}"
            th:text="${cat}"
-            th:selected="${cat == file.category}"></option>
+            th:selected="${cat == file.categoryLabel}"></option>

        </select>
        <input type="text" name="institution" class="form-control form-control-sm"
          style="width:130px" placeholder="기관명"
-          th:value="${file.institutionName}" required>
-
-        <input type="text" name="year" class="form-control form-control-sm"
-
-          style="width:64px" placeholder="연도" maxlength="4"
-
-          th:value="${file.year}">
-
-        <button type="submit" class="btn btn-primary btn-sm">재분류</button>
+
+          th:value="${file.institutionName != '기관미상' ? file.institutionName : ''}" required>
+
+        <button type="submit" class="btn btn-primary btn-sm">등록 후 재파싱</button>

        <span class="rework-result" style="display:none;font-size:12px;font-weight:600"></span>
      </form>
    </td>
  </tr>

```

10. 테스트

test/ApplicationContextTest.java

```
@@ -1,9 +1,8 @@

package com.kb.uploader;

import com.kb.uploader.job.BackgroundScheduler;
import com.kb.uploader.job.LocalScheduler;
- import com.kb.uploader.job.ReclassificationTrigger;

import org.junit.Test;
import org.junit.runner.RunWith;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.boot.test.context.SpringBootTest;
@@ -47,12 +46,8 @@ public class ApplicationContextTest {

    assertTrue("폴백이 아니라 " + scheduler.describe(),
               scheduler instanceof LocalScheduler);
}

- @Test
- public void 재분류_트리거가_빈으로_있다() {
-     assertNotNull(context.getBean(ReclassificationTrigger.class));
- }

@Test
public void Spring_자체_스케줄러는_더_이상_만들어지지_않는다() {
    // @EnableScheduling 이 남아 있으면 ScheduledAnnotationBeanPostProcessor 가
```

test/config/ConfigEnvTest.java

```
@@ -36,9 +36,14 @@ public class ConfigEnvTest {

    "mybatis.mapper-locations",
    "mybatis.type-aliases-package",
    "mybatis.configuration.map-underscore-to-camel-case",
    "upload.base-dir",
-    "reclassification.cron");
+
+    // 2026-09-23 파싱 전환: reclassification.cron(재분류 배치)이 빠지고
+
+    // 원본 보관·산출물 경로 3개가 들어왔다. 한 환경이라도 빠지면 그 환경에서
+
+    // FileStorageService 가 @Value 를 못 채워 기동이 죽는다.
+
+    "upload.userdata-dir",
+    "upload.proposal-json-dir",
+
+    "upload.rfp-md-dir");

/**
 * DataSource 계열은 환경마다 방식이 갈려서(내부망 prod 는 JNDI, 나머지는 직접 접속)
 * 여기서 요구하지 않는다 - 대신 <b>둘 중 하나는</b> 있어야 한다.
```

test/service/FileStorageServiceTest.java

```
@@ -17,9 +17,11 @@ public class FileStorageServiceTest {

    private FileStorageService sut;

    @Before
    public void setUp() {
-       sut = new FileStorageService(tempFolder.getRoot().getAbsolutePath());
+       String root = tempFolder.getRoot().getAbsolutePath();
+       sut = new FileStorageService(root, root + "/userdata",
+           root + "/out-json", root + "/out-md");
    }

    @Test
    public void unclassified_저장_파일생성() throws Exception {
@@ -57,5 +59,36 @@ public class FileStorageServiceTest {

        assertTrue(moved.toString().contains("2024"));
        assertTrue(moved.toString().contains("서울대학교"));
        assertFalse(Files.exists(source));
    }
+
+    // — 2026-09-23 파싱 전환으로 추가된 API —
+
+    @Test
+    public void 원본은_userdata에_보관한다() throws Exception {
+        MockMultipartFile file = new MockMultipartFile(
+            "file", "제안서.pptx", "application/octet-stream", "내용".getBytes());
+
+        Path saved = sut.saveOriginal(file, "제안서.pptx");
+
+        assertTrue(Files.exists(saved));
+        assertTrue(saved.toString().contains("userdata"));
+    }
+
+    @Test
+    public void 산출물_중복이름은_숫자를_붙인다() throws Exception {
+        Path dir = sut.getProposalJsonDir();
+
+        Path first = sut.writeOutput(dir, "out.json", "{}".getBytes());
+        Path second = sut.writeOutput(dir, "out.json", "{}".getBytes());
+    }
}
```

```

+     assertEquals("out.json", first.getFileName().toString());
+     assertEquals("out_2.json", second.getFileName().toString());
+ }
+
+ @Test
+ public void 파일명에서_경로구분자를_떼어낸다() {
+     assertEquals("보고서.pdf", FileStorageService.safeFileName("C:/temp/보고서.pdf"));
+     assertEquals("보고서.pdf", FileStorageService.safeFileName(".././보고서.pdf"));
+     assertEquals("unnamed", FileStorageService.safeFileName(null));
+ }
+
+ }

```

test/service/InstitutionServiceTest.java

```

@@ -17,19 +17,17 @@ import static org.mockito.Mockito.*;

public class InstitutionServiceTest {

    private InstitutionMapper institutionMapper;
-    private ClassificationService classificationService;

    private UploadedFileMapper uploadedFileMapper;
    private InstitutionService sut;

    @Before
    public void setUp() {
        institutionMapper = mock(InstitutionMapper.class);
-        classificationService = mock(ClassificationService.class);

        uploadedFileMapper = mock(UploadedFileMapper.class);
        when(uploadedFileMapper.findByStatus("UNCLASSIFIED")).thenReturn(Collections.emptyList());
-        sut = new InstitutionService(institutionMapper, classificationService, uploadedFileMapper);
+        sut = new InstitutionService(institutionMapper, uploadedFileMapper);
    }

    @Test
    public void findAll_저장소_목록_위임() {

```

10. 지울 파일

아래 파일은 더 쓰지 않는다. 지우지 않으면 컴파일이 깨진다(없어진 클래스를 참조한다).

파일	이유
service/ClassificationService.java	폴더 이동 분류가 사라졌다
service/FileParserService.java	파일명 규칙 파싱이 사라졌다
dto/ParsedFileName.java	위와 한 쌍
job/ReclassificationJob.java	자동 재분류 배치 폐지
job/ReclassificationTrigger.java	자동 재분류 배치 폐지
controller/KGI12000\$FileStatusList.java	KGI12000\$ParseStatusList 로 대체
templates/file-status.html	parse-status.html 로 대체
test/controller/UploadControllerTest.java	옛 업로드 API 기준
test/service/FileUploadServiceTest.java	옛 업로드 API 기준
test/service/ClassificationServiceTest.java	대상 클래스 폐기
test/service/FileParserServiceTest.java	대상 클래스 폐기
test/job/ReclassificationJobTest.java	대상 클래스 폐기
test/job/ReclassificationTriggerTest.java	대상 클래스 폐기

남겨 둘 것. job/BackgroundScheduler · LocalScheduler · TimerManagerScheduler · config/SchedulerConfig 는 지금은 쓰는 곳이 없어지지만 WebLogic TimerManager 추상화라 내부망 고유 자산이고 테스트도 있다.
KGI12500\$CleanupDirs 도 옛 classified/ 정리에 필요하니 남긴다.

11. 검증

11-1. 빌드

1. `mvn -B clean test -D MapperDialectTest` 가 통과해야 두 방언 XML 이 모두 성립한다.
2. 기동 후 로그에 `Started UploaderApplication` 이 보일 것.

11-2. DB

확인	기대
DDL 의 검증 쿼리(컬럼명 조회)	새 컬럼 7개가 정상 한글
<code>SELECT COUNT(*) FROM TSKGIAF01 WHERE 파싱상태구분 = '03'</code>	오류 없이 0 이상
파일 1건 업로드 후 그 행 조회	문서종류구분·파싱상태구분·파싱산출물경로내용·파싱일시가 채워져 있을 것. null 이면 resultMap 누락이다

11-3. 화면·API

경로	확인
GET /	KPI 4개(전체·파싱 성공·파싱 실패·기관 미분류)
POST /upload/proposal	pptx → 원본/전처리/입찰제안서_PPT/ 에 JSON
POST /upload/rfp	hwp-pdf → 전처리/RFP/파싱후/ 에 md
GET /parse-status	필터 3종, 실패 사유, 재파싱 버튼
GET /parse-status/{id}/output	한글 깨짐 없이 보일 것
GET /file-status/classified	기관 미분류 목록 → 기관 지정하면 등록 + 재파싱
GET /api/proposals · /api/rfps	목록 JSON
GET /api/rfps/{id}?format=text	md 원문

외부망에서 확인한 결과(참고). 실제 HWP(서울에너지공사 금고 지정 제안요청서)로 업로드 → 파싱 성공 → 기관 등록 후 재파싱까지 확인했다. TSKGIAF01 에 문서종류 02, 파싱상태 02, 문서일자 20260801 이 들어갔고 요약 md 의 사업명·공고일·사업기간도 맞다. 테스트 64건 통과.

12. 함정 모음 — 실제로 겪은 것들

일자	증상	원인과 교훈
09-22	Unknown column '분류상태구분'	인코딩이 어긋난 채 DDL 실행 → 컬럼명 모지바케. UTF-8 도구로 실행하고 HEX 로 확인
09-09	DB 에는 값이 있는데 화면은 빈칸	resultMap 누락. 게다가 읽어서 그대로 UPDATE 하는 경로에서 null 이 되써 진다
09-08	한쪽 DB 에서만 SQL 이 늡음	databaseId 분기를 한쪽만 수정. 항상 두 벌을 같이, MapperDialectTest 로 검사
09-23	공고일이 2027-01-01	"재공고" 가 든 줄에 걸려 다음 줄의 약정기간 시작일을 가져옴 → 레이블 기반으로 수정
09-23	사업명이 사 업 개 요	별지 서식의 표 머리글을 "레이블 값" 으로 착각 → 머리글 판정 추가
상시	없는 클래스를 참조한다는 오류	target/ 에 남은 옛 .class. 클래스를 지우는 작업 뒤에는 mvn clean
상시	한글 폼값·파일명이 깨짐	server.servlet.encoding.* 와 server.tomcat.uri-encoding 을 넣을 것(4장)

작성: Claude Code 세션 (2026-09-23) · 기준 2ebfcda → 1515821 · 미해소 항목은 GGFileserver/uploader/TODO/TODO_modify.md